

Storing and Analyzing Large Graphs

What if all you need is a database

by

Puneet Mehrotra

M. Science, University of British Columbia, 2019

B. Engineering, Birla Institute of Technology and Science, 2013

A PROPOSAL SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR ADVANCEMENT
TO CANDIDACY FOR THE DEGREE OF

Doctor of Philosophy

in

THE FACULTY OF GRADUATE AND POSTDOCTORAL STUDIES
(Computer Science)

The University of British Columbia
(Vancouver)

July 2024

© Puneet Mehrotra, 2024

The following individuals certify that they have read, and recommend to the Faculty of Graduate and Postdoctoral Studies for acceptance, the candidacy proposal entitled:

Storing and Analyzing Large Graphs

submitted by **Puneet Mehrotra** in partial fulfillment of the requirements for the degree of **Doctor of Philosophy** in **Computer Science**.

Examining Committee:

Margo Seltzer, Computer Science, UBC

Supervisor

Alexandra Fedorova, Electrical and Computer Engineering, UBC

Supervisory Committee Member

Thomas Pasquier, Computer Science, UBC

Supervisory Committee Member

Abstract

Graph-structured data is common, and data from many domains can be naturally expressed as graphs. As large graphs become ubiquitous, there is an increasing interest in storing them and processing them efficiently at scale. Graphs are rarely stored natively; most graph data is extracted from data resident in some persistent database and then transformed into an in-memory representation that can be used for graph-wide statistics. State-of-the-art graph processing systems are built around these extract-transform/preprocess-load (ETL) pipelines; this process must be repeated every time the data changes. Traditional relational databases have been avoided for running whole-graph analytics for many reasons ranging from an unintuitive interface to poor performance. Graph databases work well for storing and managing dynamic graph data; however, they are often overlooked as a candidate for long-running iterative graph algorithms. This is the central dichotomy of graph data management: graph databases allow for persistent storage of graphs in a native representation, but they are outperformed by specialized graph processing systems that require a costly ingest and preprocessing phase before executing efficient graph analytics queries.

This thesis claims that it is possible to store and manage dynamic graph-structured data in a performant and extensible way. We designed and implemented a transactional graph database system based on a production-ready general-purpose key-value storage engine, providing an API that lets users interact with their data efficiently. Our system is extensible and allows the user to store multi-layer property graphs and gives them a choice of three different storage formats. The API is flexible enough that users can express their graph computations in terms of vertex or edge operations; the system itself does not restrict the paradigms that can be used. Our system is designed for a single-node, single-socket environment, and we show that high performance on graph analytics tasks can be achieved without scaling to a cluster. Additionally, we show that our system scales to large graphs ~~that scale to~~ billions of edges and can handle transactional and analytical workloads simultaneously. ~~with~~

Our architecture provides the best of both worlds: the data management capabilities one expects from a database, with the performance of a specialized graph processing system with none of the ETL or preprocessing costs involved in those solutions.

Contents

Abstract	ii
List of Tables	v
List of Figures	vi
Glossary	viii
1 Introduction	1
1.1 State of the Art: What’s good?	2
1.2 State of the Art: What needs to be changed?	3
1.2.1 Single-node optimization before scale-out	3
1.2.2 Best-of-both worlds design for mixed OLAP and OLTP workloads	3
1.2.3 In-Database Analytics	4
1.3 What have we done?	4
2 Background	6
2.1 Graph Models	6
2.2 Graph Workloads	7
2.2.1 Types of Graph Queries by Scope	8
2.3 Graph Representations	10
2.3.1 Adjacency Matrix	10
2.3.2 Adjacency Lists	11
2.3.3 Compressed Sparse Row	12
2.3.4 Persistent Graph Representations	13
3 Design Space of Graph Data Management Systems	14
3.1 Property Graphs vs. Structural Graphs	14
3.2 Static Graph Processing Systems	15
3.2.1 Programming Models vs Computation Models	15
3.3 Dynamic Graph Processing Systems	18
3.4 Distillation of the Design Space	18

4	Flexograph Architecture	20
4.1	What should a Graph Database Provide?	21
4.2	Architecture	21
4.2.1	Graph API	23
4.2.2	Internal Graph Operators for Defining Higher-level Access Primitives	23
4.2.3	Graph Representations	25
4.3	Implementation	25
4.3.1	Storage Engine	25
4.3.2	Anatomy of a Flexograph Application	26
4.3.3	Graph Representations	27
4.3.4	Internal Graph Operators	29
4.3.5	Bulk Loader	32
4.3.6	Preprocessing Graphs for Bulk Loading	32
4.3.7	Transactional Bulk Loader	33
4.3.8	Modified WiredTiger Bulk Loader	33
5	Evaluation	34
5.1	Research Questions	34
5.2	Datasets and Benchmarks	35
5.2.1	Modifications to GAPBS	35
5.2.2	Datasets	36
5.3	Experimental Setup	36
5.4	Systems for Comparison	36
5.5	Microbenchmarks	37
5.6	Preliminary Results	38
6	Storing Property Graphs	39
6.1	Example Property Graph Schema	39
6.2	Additional API calls for property and label management	40
6.3	Property Storage: Embedded vs. Split	40
6.4	Label Storage	41
6.4.1	Duplicating Representations	43
6.4.2	Embedding Attributes	43
6.4.3	Vertex Re-labelling	44
6.5	Research Questions	44
7	Future Work: Specialized Indexes on Graph Data	46
8	Timeline	47
	Bibliography	49
9	Appendix	57
9.1	PageRank Kernel Implementation using GraphAPI	57

List of Tables

2.1	Summary of graph workloads and their access patterns	9
4.1	Graph API interfaces that are implemented by each representation in Flexograph. These interfaces handle the structure of the graph; interfaces for metadata and property updates are introduced in Chapter 6	24
5.1	The graphs used for the evaluation with their number of vertices and edges, and the size of the uncompressed dataset.	36
5.2	Systems to which we compare to evaluate the performance of Flexograph on structural graphs . The systems are divided into four categories: Out-of-core(OOC) Analytics, In-memory Analytics, Dynamic graph stores, and Graph Databases.	37
5.3	Preliminary benchmark performance data using the PageRank and Triangle Counting algorithms on the cit-Patents graph. The times are reported in seconds. The preprocessing time mentioned for Flexograph is the time it takes to open the connection and warm the cache.	38
6.1	Graph API extensions to manage properties and labels in the graph.	41

List of Figures

2.1	A Property Graph with two node labels, "User" and "Organization", and three edge labels, "friends with", "follows", and "works at". The nodes and edges have properties associated with them.	7
2.2	Graph workloads can be classified into OLTP and OLAP workloads based on the scope of the query. OLTP workloads are typically point or neighborhood queries, while OLAP workloads are whole-graph analytics tasks.	8
2.3	A small directed graph with nine nodes and ten edges (a) and the corresponding 9×9 adjacency matrix (b).	10
2.4	soc-Epinions1 is a sparse, real-world graph that follows a power-law degree distribution. The adjacency matrix is memory inefficient. The power-law distribution shows that a few vertices have a high degree (hub nodes), while most vertices have a low degree.	11
2.5	The adjacency list and CSR representations of the sample graph shown in Figure 2.3a	11
3.1	Design Space of Graph Data Management Systems	16
3.2	Splitting the space of static graph processing systems based on the programming model (x-axis) and the computation model (y-axis). The systems marked with a blue X are hybrid: Mosaic [58] is a hybrid vertex-centric/edge-centric system, while GraphLab [57] and Hsync [94] are hybrid in their computation model and switch between synchronous and asynchronous modes of computation. . . .	17
3.3	Distillation of the design space of graph data management systems. Please refer to Section §3.3 for an explanation of how the systems listed in the dynamic but not transactional category differ from one another.	19
4.1	A typical application that uses WiredTiger (WT) resembles this diagram. The application has a single connection to the database, and each thread in the application has a private session object. Each thread can create private cursor objects using the session object to access the data.	26
4.2	Graph Representations supported by Flexograph. Each representation also maintains a metadata table (d) to store the graph properties, such as maximum and minimum vertex ID, number of vertices and edges, and whether the graph is directed or weighted. u in this diagram denotes a serialized byte array, I denotes an unsigned integer for storing degrees, and i denotes a signed integer used for edge weights.	28
4.3	Inheritance diagram for the iterator classes	30
6.1	The data schema used in the LDBC Financial Benchmark [71]. The schema consists of 5 vertex types and 9 edge types. Each vertex type has a set of properties, and each edge type has a set of properties. The schema is a property graph schema that defines the types of vertices and edges and the properties associated with them. The figure has been grabbed from [71, Figure 3.1]	40

6.2	Two strategies for property storage in the Adjacency List representation. The first strategy stores properties inline with the vertex or edge (Figure 6.2a and Figure 6.2b), while the second strategy stores properties in separate column groups (Figure 6.2c and Figure 6.2d).	42
6.3	Two strategies for property storage in the EdgeKey representation. The first strategy stores properties inline with the vertex or edge (Figure 6.3a), while the second strategy stores properties in separate column groups (Figure 6.3b).	42
6.4	Assume that we are maintaining two Edge Key representations, one for Accounts and the other for Businesses. While iterating through the neighborhood of the Account Edge Key table, we have to repeatedly query into the Business Edge Key table. The Adjacency List case is similar.	43
6.5	Using the “source type - destination type” attribute eliminates the jumping between tables but creates non-sequential accesses in both representations.	44
6.6	Partitioning and re-assigning the vertex id space according to vertex types gives us sequential access without having to scan extra entries in the neighbourhood.	45
8.1	Timeline between now until graduation	48

Glossary

RDBMS Relational Database Management System

OLTP Online Transactional Processing

BI Business Intelligence

OLAP Online Analytical Processing

HTAP Hybrid Transactional and Analytical Processing

ETL Extract, Transform, Load

HGTAP Hybrid Graph Analytics and Processing

CSR Compressed Sparse Row

CRUD Create, Read, Update, Delete

API Application Programming Interface

WT WiredTiger

GAPBS GAP Benchmark Suite

Chapter 1

Introduction

New data are being generated rapidly, and there is an ever-present and growing need to store and analyze them efficiently. Much of this data has some inherent schema or structure, and relational databases are a natural choice for efficient storage. Relational Database Management System (RDBMS) have been the backbone of data management for six decades and have been repeatedly extended to service new and emerging use cases [28, 50, 85, 86, 96]. RDBMSs are well-suited for storing and querying structured or semi-structured data on workloads with high volumes of small updates and stringent consistency requirements; these tasks are called Online Transactional Processing (OLTP) workloads. Larger scale data analysis frequently requires full dataset scans and extensive join processing, and these data warehouse or Business Intelligence (BI) workloads are called Online Analytical Processing (OLAP). Several relational OLAP databases and data warehousing solutions exist to enable such enterprise scale workloads [15, 28, 46, 73].

However, in some applications, the relationships captured in the data are themselves a fundamentally important object of study. In myriad applications ranging from social network analysis to brain imaging, these relationships form a graph, and that graph itself is important to study and understand. For example, a social network maintains information about every user in the network, such as their name, occupation, address, interests, etc. Meta (Facebook) uses a relational database to store this social network data [31]. However, the "network" part of a social network represents important relationships such as "follows" or "likes". These relationships form a large graph. Each vertex of this graph might have associated properties (i.e. schema from the relational table for users), and each edge might also have properties, but almost certainly has a type (or label), that describes the relationship that the edge represents, associated with it. Graphs in which the nodes have properties and the edges have labels are called Labeled Property Graphs [26].

Graph analysis tasks typically access the whole (or a significant part of the) graph. If we store the graph only in its relational form, relations (edges) are encoded as joins, and graph traversals turn into recursive self-joins [17]. As a result, RDBMSs do not perform well for graph analytics tasks, as computing these recursive self-joins is expensive [27].

In response to this mismatch between graph analysis tasks and relational databases, we have witnessed an explosion in special-purpose systems for storing and processing graph-structured data. Specialized graph databases such as Neo4j [13] and ArangoDB [10] follow in the tradition of relational databases, providing a schema-oriented organization and a high-level query language. These systems work well for OLTP workloads but frequently exhibit suboptimal performance on OLAP tasks. In contrast, a second class of graph processing systems, such as Pregel [61], GraphX [45], PowerLyra [35], GraphChi [53] and Ligra [82] are specifically designed to provide outstanding OLAP performance but may not provide OLTP-style query capabilities.

OLTP queries on graphs are similar to OLTP queries on more conventional data; for example, in a social network, a user might wish to see the latest posts and replies on their favorite ongoing Twitter feud [48]. Such *interactive* queries are latency-sensitive, with many tasks, such as fraud detection in financial networks, demanding 20 ms latencies [72].

Analytical workloads on graphs (Graph OLAP) are queries about the network that can be answered only through a "whole-graph" analysis. For example, the recommendation teams at Twitter run PageRank and topic similarity tasks on the user-follow graph to measure the influence of users and recommend new users to follow [88]. The health teams at Twitter and Facebook run multi-account and fake account detection algorithms on the network graphs to identify and remove fake accounts [30]. These tasks must interact with large parts of the graph, if not the whole graph, and will typically take longer to finish.

This thesis presents the design of a scalable and efficient hybrid "best of both worlds" graph data management system that stores graphs in a database structure but can handle both analytics tasks and transactional queries using analytics-friendly formats. Designing such a system needs a close examination of the state-of-the-art to identify extant designs that are aligned with our design goals (Section 1.1). We also identify a few anti-patterns we wish to avoid (Section 1.2) while designing our hybrid system. Together with our novel contributions in graph state management, principled persistent layout design, and thread scheduling, Flexograph can achieve performance comparable to in-memory graph processing systems while also being competitive with graph databases.

1.1 State of the Art: What's good?

State-of-the-art systems typically assume that graph databases and graph processing systems run on a distributed collection of nodes. We, instead, start with the assumption that the best distributed system is built from the best single-node system. Based on this assumption, we identify a set of requirements for building such a single-node graph Hybrid Transactional and Analytical Processing (HTAP) system.

1. **Efficient out-of-the-box algorithms and a rich, useful API for expressing computations:** Using efficient algorithms is essential for fairly evaluating the system and establishing areas that need improvement [25, 64, 65]. Additionally, it is useful to provide an intuitive and flexible API that allows users to write custom algorithms.
2. **Using well-designed data structures that can be parallelized without incurring severe penalties from locking overheads** is critical in designing parallel graph processing systems that leverage today's multicore compute infrastructure and fast storage devices.
3. **Using data layouts that maximize sequential access to the graph are preferred** and painstakingly designed. This has been the central theme for much research in semi-external graph processing¹, and every system has its own clever scheme to address this problem for its API and computation model. Creating specialized data layouts requires preprocessing the graph, which is a significant overhead relative to the cost of running many computations. Preprocessing costs are rarely amortized.

We build upon these ideas to create Flexograph.

To eliminate the overhead of compare-and-swap or locking-based update propagation, we use a lock-free implementation for key data structures in our code.

¹In semi-external systems, vertex, and algorithmic state is maintained in RAM and the graph edges are read from the storage device

Flexograph supports two different data layouts that each have their own benefits and drawbacks, which we analyze in §2.3. We provide a detailed comparative analysis of these data layouts and hints for choosing a data layout based on the read-write mix an application might experience.

Contrary to the popular practice of building a custom storage engine for accessing the graph, we show that building a graph database on a general-purpose, production-quality key-value store provides performance comparable to other persistent graph processing systems. And, when we leverage the key-value store cache, we provide in-memory performance comparable to that of main-memory graph processing systems.

1.2 State of the Art: What needs to be changed?

Graph data management has a fork in the road: one set of systems focuses on high-throughput whole-graph analytics, while the other focuses on low-latency OLTP-style processing; data is transferred between these systems via extract-transform-load pipelines. Flexograph is designed to find a best-of-both-world solution in this landscape that reduces cross-silo data movement and preprocessing costs. We base the Flexograph design on a few key principles that we lay out in the following subsections, namely, single-node optimization, in-database analytics, and bridging the gap between graph OLAP and graph OLTP systems.

1.2.1 Single-node optimization before scale-out

Past research has conclusively shown that premature distribution of tasks based on poor single-node baselines is doomed to perform badly. McSherry et al. [64] demonstrated that several popular distributed GPS evaluated their scalability by comparing, not against an efficient single-threaded implementation, but by showing how the run time changes as a function of the number of threads. When McSherry compared these scalable systems to an efficient sequential implementation, he revealed that some systems could not match “laptop performance” regardless of how many machines were used! This stunning result led to several attempts at benchmark standardization efforts for GPS [12, 23, 32, 65]. Each proposal identified the need to use more performant algorithms and report more comprehensive performance metrics. Considering that distributed graph processing is fundamentally inefficient due to the communication costs, the main benefit of using a cluster for graph processing is that the aggregate RAM might be sufficiently large to enable processing graphs entirely in memory, even if their size is larger than the memory of a single machine. The Waterloo survey on the state of graph processing [77] reveals that large graphs are ubiquitous, and several of their respondents reported operating on uncompressed graphs larger than 1 TB; the exact upper limit on the size of these graphs is unknown [77, Section 3.2]. While servers with more than a few terabytes of RAM exist, they are expensive and, as we and others have shown, unnecessary. Scale-out systems built from semi-external graph processing systems should provide better price performance than a distributed graph/data processing system. We want to design and optimize a single-node, single-socket graph database before requiring distribution.

1.2.2 Best-of-both worlds design for mixed OLAP and OLTP workloads

As reported in section 2.3, many graph processing systems require expensive pre-processing steps to construct a suitable on-disk data layout that allows fast sequential access to the graph. Time spent preprocessing the graph is rarely amortized; it varies by the workload’s freshness requirements and the rate at which new snapshots or increments need to be analyzed.

A better alternative is to natively store the graph in a format similar to or close to the in-memory representation required for efficient access. Graph databases do this but are optimized for transactional point queries rather than

full-graph analytics. Neo4j [13], for instance, stores all graph data on disk as "linked lists of fixed size records" [66]. A linked-list representation of graphs is great for connectivity queries but is bound to cause catastrophic random access overhead when running whole graph analytics. Newer research systems such as GraphFlowDB [47] provide a more methodical analysis of design decisions required to store a property graph in memory. Still, they do not persist or evaluate the system on whole-graph analytics benchmarks. Storing a graph larger than available RAM requires careful persistent and in-memory data layout co-design, which performs well on both OLAP and OLTP-style workloads.

1.2.3 In-Database Analytics

We examined the use of specialized graph processing systems for analytics tasks and know they are typically not transactional and do not support point queries. Graph databases are transactional and perform well on point queries but are orders of magnitude slower than specialized graph processing systems [69] on whole-graph analytics tasks. Therefore, state-of-the-art graph data management has evolved around a split system design: graph database systems for point queries and specialized high throughput graph processing systems for whole-graph analysis tasks. This leads to a freshness mismatch between the data stored in the original data store, what's available for point-query analysis following ingestion into a graph database, and what's available for batch analysis in a specialized graph processing system. The split system design also causes the creation of data silos which need constant Extract, Transform, Load (ETL) and data movement to maintain consistency.

These problems have an equivalent in the relational database world: the constant data transformation needed to maintain consistency between a row-store and a column store [22], and the subsequent development of HTAP databases is a promising solution. Similar solutions must also be developed for analyzing graphs, and recent commercial graph databases, such as GART [80], have started offering solutions for real-time graph analytics workloads. GART stores the extracted graph in memory since it has been designed for latency and freshness-sensitive tasks, such as fraud detection, and not for further downstream batch processing tasks we target. Graph HTAP systems should be designed to handle efficient real-time analytics and more intensive whole-graph processing tasks.

1.3 What have we done?

This thesis presents the design of a flexible and performant Hybrid Graph Analytics and Processing (HGTA) database system that performs better than custom-created graph processing systems on analytics tasks and is competitive against graph databases on transactional point queries.

Our system, Flexograph, has been designed for a single-node setting and provides a fully transactional API for updating and querying a graph. We provide snapshot-level read-only iterators for fast serial reads for analytical tasks. Flexograph has been designed to eliminate any preprocessing tasks needed before running analytics and provides two layouts we designed to get strong sequential iterator-based read performance common in analytics tasks. The API is flexible enough to let users write algorithms as vertex or edge programs; we demonstrate our performance by modifying GAP-BS to use our API and demonstrating that our analytics tasks have robust performance compared to other semi-external and in-memory systems when factoring in their pre-processing times.

Our system handles property graphs, and we evaluate the benefits of storing the graph's structure independently from their properties and contrast it with the choice to store them together.

The contributions of this thesis are:

1. The design of a Graph HTAP database takes a "best-of-both-worlds" approach to deliver robust performance on whole-graph analytics tasks and transactional queries without the need for constant preprocessing or ETL.

2. We present a principled analysis of analytics-friendly data layouts and how we use a mature key-value store (WiredTiger [9]) as the storage engine to store and access the graph. The API provided by Flexograph allows the user to do fast neighborhood scans by using scan cursors and provides an API for persisting the algorithmic state of graph processing tasks to bootstrap future rounds of computations.
3. Flexograph provides a multi-threaded bulk-insertion tool for quickly loading the initial graph into the database. Most other bulk-insertion tools require a single-writer to the database due to consistency requirements; we relax these requirements for fast bulk insertion.
4. We provide a comprehensive performance analysis of Flexograph against graph databases and graph processing systems (in-memory, semi-external, and dynamic) to show that we are competitive in all cases, but with lower data migration cost due to low/no preprocessing and ETL costs.

The remainder of this thesis proceeds as follows: In [Chapter 2](#), we provide the background that establishes concepts central to our work. We begin by describing the data structures used to represent graphs efficiently in memory using layouts that maximize sequential access, particularly the most popular layouts: Compressed Sparse Row (CSR) format (§2.3.3), [Adjacency Lists \(§2.3.2\)](#), and [Edge Logs \(§2.3.1\)](#). We discuss the design of data layouts in graph processing systems in [Section 2.4](#) and graph databases in [Section 2.5](#). Following that, we examine how semi-external systems create on-disk layouts that allow efficient access to the graph. We then turn our focus towards graph databases and examine the transactional queries for which they are optimized. With this context in place, we contrast graph databases and graph processing systems, establishing the central dichotomy of graph data management and lay the foundation to discuss our hybrid graph transactional and analytical system design.

[Chapter 4](#) presents the design of Flexograph. We begin by describing our graph API and then introduce the three layouts for which we provide that API. Next, we describe the iterators we provide for efficiently accessing graph elements: nodes, edges, and neighborhoods of nodes. We conclude with a description of work assignment and thread scheduling for reduced-conflict writes to the graph. Finally, we discuss the optimizations we provide for maintaining the algorithmic state and using results from previous computations to jumpstart new rounds of computation.

[Chapter 6](#) describes how we extend Flexograph to store property graphs [20]. We begin with a description of the design alternatives available to us for storing properties with graph objects and describe the cost of using these two approaches. We compare these two designs with each other and against other dynamic graph processing systems and graph databases on LDBC queries, and show that we can perform similar to our competitors.

We discuss future work in [Chapter 7](#) and present a timeline for the remainder of the PhD in [Chapter 8](#).

Chapter 2

Background

This chapter covers the background information necessary to understand the research presented in this thesis. We begin by defining graphs and the Property Graph Model, which is the most common graph model used for modeling real-world data. We then discuss the different types of graph workloads and study the access patterns they induce on the graph. We conclude the chapter by examining the different data structures used to support these workloads.

2.1 Graph Models

A graph G is defined as a pair $G = (V, E)$ of sets V and E such that $E \subseteq V \times V$. V is the set of *vertices* (also called “nodes”) of the graph G , and E is the set of *edges* of G [39]. The vertices are the entities modeled by a graph, and an edge represents a relationship between the vertices to which it connects. The edges of a graph can be directed or undirected. In a directed graph G , an edge $e = (u, v) \in E$ is an ordered pair, where u is called the “source” vertex and v is the “destination” vertex. If G is undirected, then an edge $e = (u, v) \in E$ is a *set* of two vertices. A weighted graph has a “weight” associated with each edge. Formally, a weighted graph G is modeled as a triple (V, E, w) , where $w : E \rightarrow \mathbb{R}$ maps edges to weights.

This formulation of a graph captures only the structure of the graph; for the remainder of this dissertation, we will refer to this as a *Structural Graph*. Structural graphs are appropriate for graph theoretic studies and analytics tasks that examine the graph’s topology and connectivity (e.g., finding shortest paths, computing centrality measures, and detecting communities). However, real-world graphs model entities that have a “schema,” i.e., they have types and properties associated with them. Capturing the additional information allows us to perform more complex queries on the graph, such as finding all users who are friends with a particular user or finding all users who have a particular property.

The augmented graph model that satisfies the requirement to have properties and labels associated with graph elements is called the *Labeled Property Graph (or simply a Property Graph)* Model [21]. We use the term Property Graphs (PG) to refer to these graphs in the remainder of this dissertation. Each vertex and edge in a PG has labels, which are used to construct subsets of vertices and edges and are useful for filtering queries. For instance, in a social network, the nodes could be labeled “users” or “organizations,” and the relationship labels could be “is friends with” or “follows.” The concept of labels has an equivalent in the relational model: node labels are analogous to tables and the edge labels are analogous to foreign keys or join tables.

Additionally, the nodes and the edges in a PG could have an arbitrary number of *properties* (also called *attributes*) associated with them. If a relational table corresponds to a label in a property graph, then a row in the table represents a node or edge, and the columns represent the node’s properties. For instance, a node with a “user” label in a social

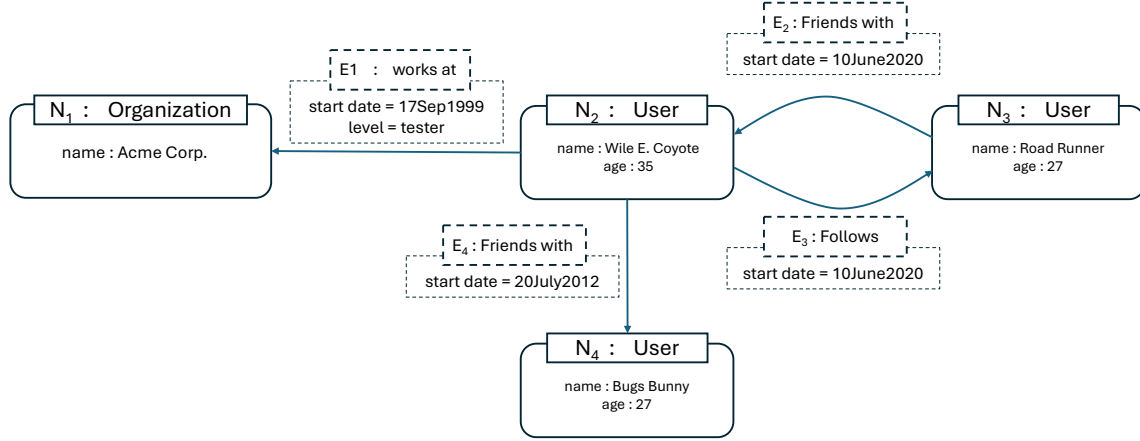


Figure 2.1: A Property Graph with two node labels, "User" and "Organization", and three edge labels, "friends with", "follows", and "works at". The nodes and edges have properties associated with them.

network graph could have properties such as "name," "age", and "location". Similarly, an edge with a "works at" label could have properties such as "start date", "level", etc. These properties are modeled as (*key*, *value*) pairs, where *key* identifies the property and the *value* is the corresponding value of that property. Figure 2.1 shows an example of a property graph with two node labels, "User" and "Organization", and three edge labels, "friends with", "follows", and "works at".

A more formal definition of Property Graphs can be found in the survey papers by Angles et al. [21] and Yakovets et al. [29]. It is important to note that structural graphs are a subclass of property graphs. Property graphs can be transformed into structural graphs by selecting nodes and edges with specific labels and removing all associated properties. For example, Figure 2.1 can be converted into a structural graph by selecting nodes labeled "user" and edges labeled "friends with" and then removing all associated properties.

2.2 Graph Workloads

In Chapter 1, we briefly explored typical relational database workloads: Online Transactional Processing (OLTP) and Online Analytical Processing (OLAP). OLTP workloads involve tasks that insert or update data in a database at high volume using transactions. These systems are optimized for a high volume of queries that are typically latency-sensitive, short-running, and small, typically modifying only a small portion of the database. In contrast, OLAP workloads are read-mostly and are used to analyze and aggregate data. While analytical processing tasks also involve high volumes, they are generally less sensitive to latency and are designed to maximize throughput.

Similarly, graph data management systems can handle both OLTP and OLAP workloads. An example of a graph OLTP workload is a neighborhood query, such as finding a user's friends in a social network graph. These queries tend to access small parts of the graph, are short-running, and transactional. Other graph OLTP tasks include inserting or updating vertices or edges, looking up nodes or edges, and other interactive queries. These typically involve immediate responses and real-time data manipulation.

In contrast, graph OLAP queries are usually long-running and complex tasks that often involve reading large portions of the graph, if not the entirety. Familiar examples of analytics workloads include running Breadth-First Search (BFS), PageRank, community detection tasks (such as triangle counting and label propagation), shortest path

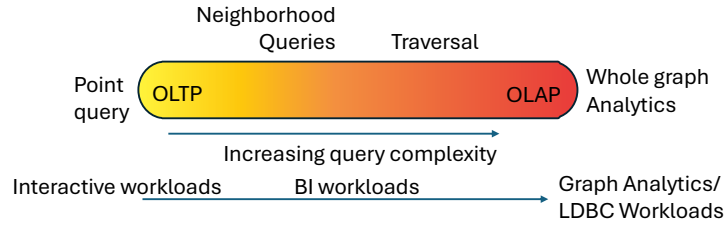


Figure 2.2: Graph workloads can be classified into OLTP and OLAP workloads based on the scope of the query. OLTP workloads are typically point or neighborhood queries, while OLAP workloads are whole-graph analytics tasks.

computations (SSSP), pattern mining, and many others [32].

To better understand the graph workloads and how they access the graph, it is helpful to classify them based on how much of the graph they access (also called the scope of the query). A recent survey by Besta et al. [26, Section 4.2.1] provides a handy classification of graph queries on graph databases. We briefly summarize their classification here to complete our description of the research landscape.

2.2.1 Types of Graph Queries by Scope

1. **Point Queries:** Point queries on a property graph involve inserting, deleting, or updating a single graph element (vertex or edge). Another typical example is to set or retrieve some associated property or to filter edges/nodes with a certain label. To run efficiently, point queries require an index on the property or label being queried.
2. **Neighborhood Queries:** Neighborhood queries are used for finding the edges incident to or emanating from a vertex. A neighborhood query finds nodes that are reachable from (or are part of the neighborhood of) a given node. These queries are essential for social network analysis, where finding a user’s friends or followers is a common task. A common way to optimize neighborhood queries is to have direct, sequential access to a node’s neighborhood.
3. **Traversals:** Graph traversals mean visiting graph elements starting from a "root" vertex and following its edges in some deterministic or random/sampling-based fashion. Graph traversals can be thought of as a neighborhood query but with selection criteria based on the label or property of the nodes or edges. Additionally, while neighborhood queries are typically limited to a fixed depth, traversals can be unbounded. A typical traversal query on a social network could be to find friends-of-friends or friends-of-friends-of-friends who work at a particular organization. Traversal queries access have some logical locality, but they might access the physical data in random order; this makes them challenging to optimize.
4. **Whole Graph Analytics:** Computations for global graph properties (such as vertex centralities, all pairs shortest paths, or clustering coefficients) that require accessing the entire graph are called whole-graph analytics. These workloads access most (often all) vertices and edges in a graph, though not necessarily all properties and labels associated with them (i.e., the structural graph). These workloads benefit from having sequential access to all vertices in the graph and sequential access to all edges in each vertex’s neighborhood.

While *graph databases* support all types of queries, they are best suited for point and neighborhood queries.

Workload	Access Pattern	Helpful Optimization
Point Query	Random access to nodes with a certain label or property	Index on labels or property
Neighborhood Queries	Accessing a node's in- or out-neighborhood	Direct, sequential access to neighborhoods
Graph Traversals	Selection on neighborhood queries	Inherently random, no clear optimization
Whole-Graph Analytics	Access all/most nodes and edges on some materialized structural graph	Sequential access to nodes and edges in each node's neighborhood

Table 2.1: Summary of graph workloads and their access patterns

Static graph processing systems are specialized to handle whole-graph OLAP workloads on structural graphs. A summarization of this classification is provided in Table 2.1.

Wherever there are workloads of broad commercial interest, a benchmarking council is looking to formalize a specification; graph data management systems are no exception. The leading benchmarking association for graph workloads is the Linked Data Benchmark Council (LDBC) [5], and they have published several benchmark specifications to standardize the benchmarking practices on graph data. LDBC provides two popular benchmarking suites: the Social Network Benchmark (SNB) [7] and Graphalytics [6].

LDBC-SNB defines two subclasses of transactional workloads: interactive and business intelligence workloads. The interactive workload is centered around "complex read queries that access the neighborhoods of a given node and update operations that continuously insert new data into the graph." [7]. Besta et al. [26] provide a breakdown of the interactive workload into short read-only queries (neighborhood lookup for a node or short traversals), complex read-only queries (traverse a more significant part of the graph and used to assess query optimizers), and transactional update queries (insert/modify/remove a node and its incoming or outgoing edges). The Social Network Benchmark also provides the LDBC Business Intelligence workloads, which gauge the system's performance under more expansive reads that fetch large parts of the graph. Typical queries include tasks that sort, order, aggregate, and derive maximum or minimum values from the data.

The LDBC Graphalytics benchmark and the GAP Benchmark [23] describe the most important classes of graph analytics kernels based on their frequency of use in applications and their diverse computational complexities. Graphalytics includes six kernels: Breadth-First Search (BFS), PageRank (PR), Weakly Connected Components (WCC), Label Propagation for Community Detection (CDLP), Clustering Coefficient, and Single-Source Shortest Path (SSSP). The GAP Benchmark Suite [25], GAPBS, provides a specification and reference implementation for BFS, PR, Triangle Counting (TC), Betweenness Centrality (BC), and Connected Components (CC).

Another notable benchmark suite is the GraphMineSuite, which contains graph mining workloads. Kernels used for graph mining are significantly more computation-heavy than the "low-complexity" kernels used in most applications and benchmarking suites such as GAPBS and Graphalytics. GraphMineSuite provides a framework for evaluating a reference implementation for graph pattern matching, graph learning algorithms, optimization problems (such as minimum spanning tree and minimum cut), and vertex ordering tasks.

2.3 Graph Representations

In §2.1, we formally defined graph-structured data. While the ultimate goal of Flexograph is to provide robust data management for property graphs, the initial focus has been on developing analytics-friendly storage solutions for structural graphs (see Chapter 4). This foundational work sets the stage for efficient handling and analysis of property graphs in subsequent phases (see Chapter 6).

We now describe how graphs are represented by introducing the most common in-memory data structures used for storing them. We begin with the most common way to represent graphs and then follow up with other, more compact, layouts designed to enable sequential access to graph neighborhoods.

For the remainder of this section, we will use the sample graph shown in Figure 2.3a to illustrate the different graph representations.

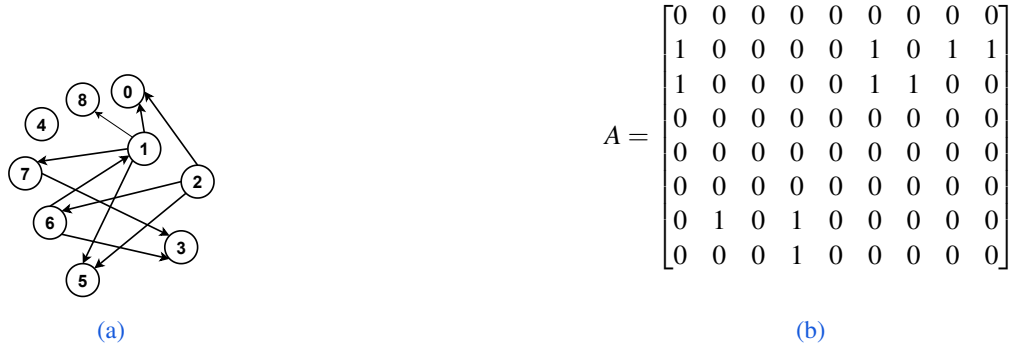


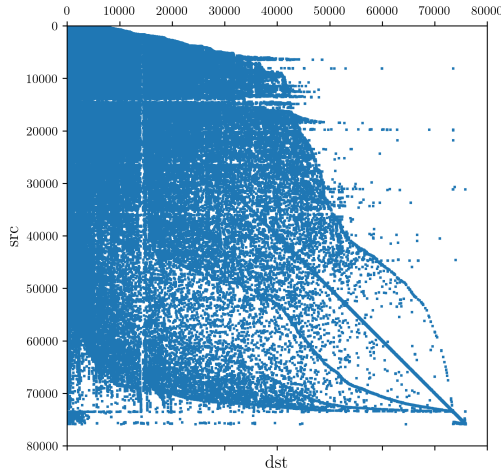
Figure 2.3: A small directed graph with nine nodes and ten edges (a) and the corresponding 9×9 adjacency matrix (b).

2.3.1 Adjacency Matrix

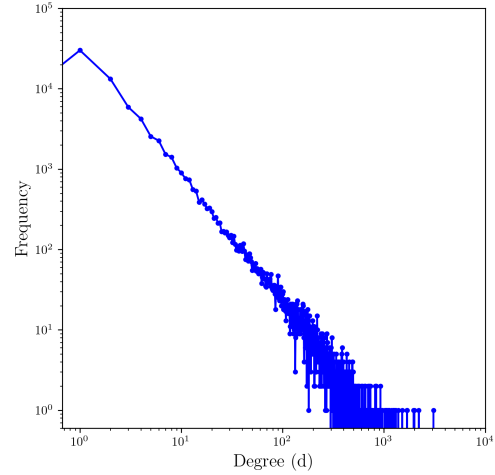
The canonical representation of a graph uses an $n \times n$ matrix, A , where n is the number of vertices in the graph; an element $a_{ij} = 1$ in A indicates the presence of an edge between vertices i and j . Depending on the matrix’s definition, edges can be directed or undirected. Hence, the edge a_{ij} can either indicate a directed edge $i \rightarrow j$ or an undirected edge $i \leftrightarrow j$. It is common to represent undirected edges between vertices i and j as two directed edges $i \rightarrow j$ and $j \rightarrow i$. The adjacency matrix representation of our sample graph is shown in Figure 2.3b.

Real-world graphs tend to follow the power law distribution [67], have small diameters, and exhibit communities [34, Section 2]. A power-law distribution of degrees leads to a few *hub* nodes with high degrees and many nodes with low degrees (i.e., the heavy tail of degree distribution). A direct consequence of the power law is that the resulting adjacency matrices are sparse and, therefore, memory inefficient. Figure 2.4 shows the adjacency matrix and the degree distribution plots for `soc-epinions1` [83], a small network with around 76,000 nodes. The white space in the adjacency matrix illustration denotes the sparsity pattern, which translates to empty cells in an adjacency matrix representation of the graph. This wasted space in an adjacency matrix leads to poor cache utilization, which is already challenging due to the random access pattern inherent in graph traversal. Better data structures that are more compressed than an adjacency matrix are essential to minimize the space requirement and maximize cache utilization.

We now examine a few memory-efficient data structures commonly used in graph processing systems and the



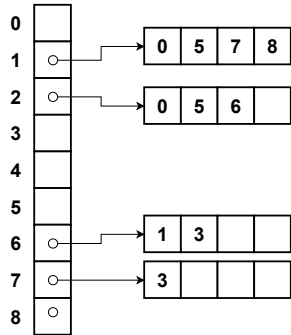
(a) The adjacency matrix for the network



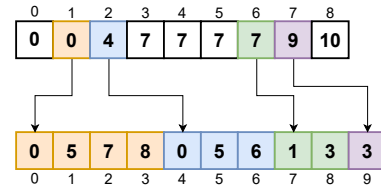
(b) Degree distribution plot for the network

Figure 2.4: soc-Epinions1 is a sparse, real-world graph that follows a power-law degree distribution. The adjacency matrix is memory inefficient. The power-law distribution shows that a few vertices have a high degree (hub nodes), while most vertices have a low degree.

cost of creating, updating, and finding neighborhoods of a node in each representation.



(a) An adjacency list representation



(b) CSR representation

Figure 2.5: The adjacency list and CSR representations of the sample graph shown in Figure 2.3a.

2.3.2 Adjacency Lists

Adjacency lists are a natural solution to the sparsity problem and memory inefficiency of adjacency matrices. We construct adjacency lists by expressing each row in the adjacency matrix as a list of a vertex's neighbors. For instance, consider the adjacency matrix shown in Figure 2.3b. To construct the neighborhood list for each node, we construct a vector containing the column index j where $a_{ij} = 1$ in the adjacency matrix. The corresponding adjacency list representation of the graph is shown in Figure 2.5a. The memory requirement for the adjacency list representation is $O(|V| + |E|)$, which is superior to the $O(V^2)$ required for an adjacency matrix.

Adjacency lists allow sequential access to a node’s neighborhood. However, a random seek is required to access the neighborhood (the vertex typically contains a pointer to the list). Adjacency lists are dynamic data structures. Inserting new edges into an unsorted adjacency list representation has an amortized $O(1)$ cost; however, if the list must be kept sorted, the cost increases to $O(\log(n))$. Typically, two adjacency lists are maintained for a graph: one for the in-neighborhood and another for the out-neighborhood.

The adjacency list representation provides $O(1)$ seeks to a vertex but often requires a random seek to access the neighborhood. Once the neighborhood is accessed, the adjacency list provides sequential access to the neighborhood. This property makes adjacency lists an excellent choice for all workloads except ones that critically depend on sequential vertex set scans. Sortledton [43] shows that this is never a problem in practice, and adjacency lists provide performance within 40% of the performance of the compressed sparse row representation for most workloads.

2.3.3 Compressed Sparse Row

Compressed Sparse Row (CSR) [76] is a compact, *immutable* data structure that uses two flat arrays to store the neighborhood lists for nodes in a graph. Assuming a non-symmetric sparse matrix A , we create two integer vectors: *vertices* and *edges*. For each $v_i \in V$, $vertices[v_i] = idx_{v_i}$; where idx_{v_i} is the index in *edges* where v_i ’s adjacency list begins. The contents of $edges[idx_{v_i}]$ to $edges[idx_{v_i+1}]$ are the sets of vertices v_j such that $a_{ij} \neq 0$.

CSRs provide $O(1)$ seeks to a vertex, and since all nodes are stored in a single array, they provide sequential access to the vertex set. CSRs also provide a sequential scan of a node’s neighborhood. These properties make CSRs an excellent choice for all workloads we discussed in §2.2.

CSR stores a graph $G = (V, E)$ in $O(|V| + |E|)$, but since it is immutable, every modification (i.e., insert or delete) requires reconstructing the edge array. The edge array might need reallocation to accommodate an incoming edge, and the old edges must be moved and shifted around the new edge. This is a serious limitation for evolving graphs, and much research has been done to create resizable CSRs for graphs [42, 59, 60, 84, 93].

Common variations of CSRs that are insertion-friendly include:

1. **In-place Updates:** CSRs can be created to allow in-place insertion and deletion of graph elements without needing a full, expensive rebuilding of the data structure. NetworKit [84] doubles the size of the initial edge array to reserve space for new edges. A variant of the CSR called the Packed CSR [93] leaves space between elements, allowing for faster insertion and deletion of graph elements at the cost of a slowdown in traversals and an increase in the space overhead.
2. **Batching updates:** Many systems batch updates and preprocess the updates to make the insertions faster. Systems such as the one described by Madduri et al. [60] batch updates, order them by vertex ID, and sort them before applying the updates.
3. **Multi-versioning and Delta Updates:** An alternative design to support fast inserts into a graph is to store all updates in a separate structure called a Delta Map. This use of delta maps allows for running graph analytics on snapshots of the graph at different times in its evolution. LLAMA [59] is one such snapshot-based graph system that supports multi-versioning and stores the delta updates as separate concurrently accessible snapshots. There are some disadvantages to using a snapshot-based system for analytics. A high ingestion rate leads to many delta snapshots since the updates get fragmented over multiple snapshots. An analytics job must read many snapshots, and this causes significant performance degradation. It is common to merge and consolidate updates into the CSR; this process is called compaction [42, 59]. It is similar to garbage collection and leads to increased latency.

2.3.4 Persistent Graph Representations

The data structures we have discussed so far are all in-memory representations of graphs. Commodity hardware has limited memory, and the size of real-world graphs can exceed the available memory. Moreover, the data in memory is volatile and is lost when the system is shut down or suffers a crash. Due to these limitations, it is essential to have persistent graph representations that can be stored on disk and analyzed efficiently.

A persistent graph representation needs to be block-based; no in-memory data structures are block-based, and it is not trivial to convert them to a block-based representation. LLAMA [59] uses a persistent CSR representation to store an evolving graph on disk and can demand-page the relevant blocks into memory for analytics.

Out-of-core graph processing systems are designed to handle graphs that are too large to fit in memory. They use a combination of in-memory data structures and disk-based data layouts to process the graph. The key design decision in out-of-core systems is to minimize the number of disk accesses and to ensure that the disk accesses are sequential. A common way to achieve this is by partitioning the edges across multiple files in a way that keeps neighborhood accesses sequential. As the reader will see, we adopt an existing storage engine to provide graph persistence. As such, we delay discussion of existing persistent representations until the Related Work section ?? (which is something I plan to do after my candidacy exam).

Chapter 3

Design Space of Graph Data Management Systems

In [Chapter 2](#), we discussed the different models used to represent graphs – property graphs and structural graphs. We also discussed the different types of workloads that can be executed on these graphs and the access patterns they exhibit. Based on these access patterns, we identified requirements for a data structure that can store graphs and discussed the different data structures used by graph data management systems.

We use our understanding of graph models (property and structural), workloads, and in-memory representations to derive a comprehensive overview of the design space of graph data management systems. We introduce a taxonomy of the design space and discuss how existing systems fit into this taxonomy.

3.1 Property Graphs vs. Structural Graphs

Our overarching characterization of the design space is defined by two features: when the system manages property graphs or structural graphs and whether the graphs under management are static or dynamic. Recall that a structural graph results from selections and projections on the labels and properties of an underlying property graph [\[74\]](#). One might select edges of a certain type to form a structural graph, for instance, a structural graph of follower relationships in a social network. The structural graph so constructed “is homogeneous, composed simply of vertices, edges and, possibly, a weight attached to their elements” [\[37\]](#). The structural graph is a useful abstraction for many graph algorithms and queries. It provides a simple graph model that can be used to reason about the graph’s structure. For instance, the structural graph can be used to compute the connectivity of the graph or to compute the influence of a node in the graph.

We assume that the property graph represents ground truth and that the structural graph is derived from the property graph. A structural graph can be materialized as needed or periodically, as is done at Facebook [\[36\]](#). These materialized structural graphs are essentially snapshots of the property graph at a certain point in time. We describe these *static structural graphs* in more detail in [§3.2](#).

In contrast, the property graph changes over time. Since the property graph is the ground truth, it is important to keep the structural graph up-to-date with the property graph. This can be done by periodically recomputing the structural graph or incrementally updating it as the property graph changes. This leads to the concept of *dynamic structural graphs*, which are updated as the property graph changes. We describe dynamic structural graphs in more detail in [§3.3](#).

3.2 Static Graph Processing Systems

Static structural graphs have been the main focus of graph processing systems research for over a decade ([18, 23, 35, 44, 45, 51, 54, 56, 61, 68, 82, 87, 95, 98, 100, 100]). These graphs are primarily used for analytics and graph mining tasks focusing on uncovering statistical properties and discovering patterns. The static nature of these graphs implies that there is no consistency requirement between different snapshots that these systems are designed to process. Each snapshot is produced through a batch ETL process and is (often) processed independently of other snapshots. On the other hand, in a graph database, the graph is materialized from the property graph, and that is the "extract" step when creating structural graphs.

Once the static GPS has a snapshot to process, it must first *preprocess* the graph to construct the data structures needed to execute the graph algorithms. The preprocessing step is often the most time-consuming in the graph processing pipeline. The preprocessing step involves reading a graph in a data exchange format (often a CSV file) and using that to construct and populate a suitable data structure to hold the graph. This requires a pass over the graph and can take significantly longer to finish than the analytics task itself (For example, GraphChi takes longer to preprocess the graph than it takes to run Triangle Counting or PageRank Table 5.3).

If the graph data structures obtained after the preprocessing are small enough, they can be stored and processed in memory. Graph processing systems designed for shared-memory, multi-core machines are usually optimized for in-memory processing. Several systems have been optimized for this design point, for example, Gemini [100], GraphMat [87], Ligra [82], Galois [70], GPOP [54], and GraphLab [56].

If the graph structures are too large to fit in memory on a single machine, they are partitioned across multiple machines. Examples of distributed graph processing systems include GraphX [45], Pregel [61], PowerGraph [44], Distributed GraphLab [56], and PowerLyra [35]. For the purpose of our taxonomy, we collectively refer to shared-memory multi-core systems and distributed (in-memory) systems as **in-memory graph** processing systems.

Over time, researchers noticed that the communication overheads in distributed graph processing systems were high, leading them to underperform when compared to single-threaded systems running on a commodity laptop [64]. The need to build systems that can scale to large graphs that do not fit in RAM on a single machine, coupled with the underperformance of distributed graph processing systems, led to the development of out-of-core systems. These systems are designed around the idea of caching the most frequently accessed data structures in memory and spilling the rest to disk. We call these systems **out-of-core graph** processing systems. The on-disk data layout is designed to leverage sequential access to edge data to minimize I/O overheads. Most systems achieve this by some form of edge-list partitioning such that the neighbors (either in- or out-neighbors) of a vertex are stored contiguously on disk. Examples of out-of-core graph processing systems include GraphChi [53], X-Stream [75], GridGraph [99], V-part [41], Wonderland [97], CLIP [18], FlashGraph [98], and Blaze [51]. These two classes of graph processing systems – in-memory graph processing systems and out-of-core graph processing systems comprise the *third classification feature* of out design space of graph data management systems.

The static structural graph processing systems can be seen in the bottom left corner of the design space in Figure 3.1. Figure 3.1 shows the memory-based/out-of-core axis as a gradient, where the out-of-core systems are shown in yellow shades, while the in-memory systems are shades of red. This classification is based on the observation that most out-of-core systems can act like an in-memory system when the graph is small.

3.2.1 Programming Models vs Computation Models

The design space of static graph processing systems can be further subdivided based on the programming and computation models for which they are designed. Several survey papers [49, 63] discuss this classification in far more detail; we present a brief overview here for completeness.

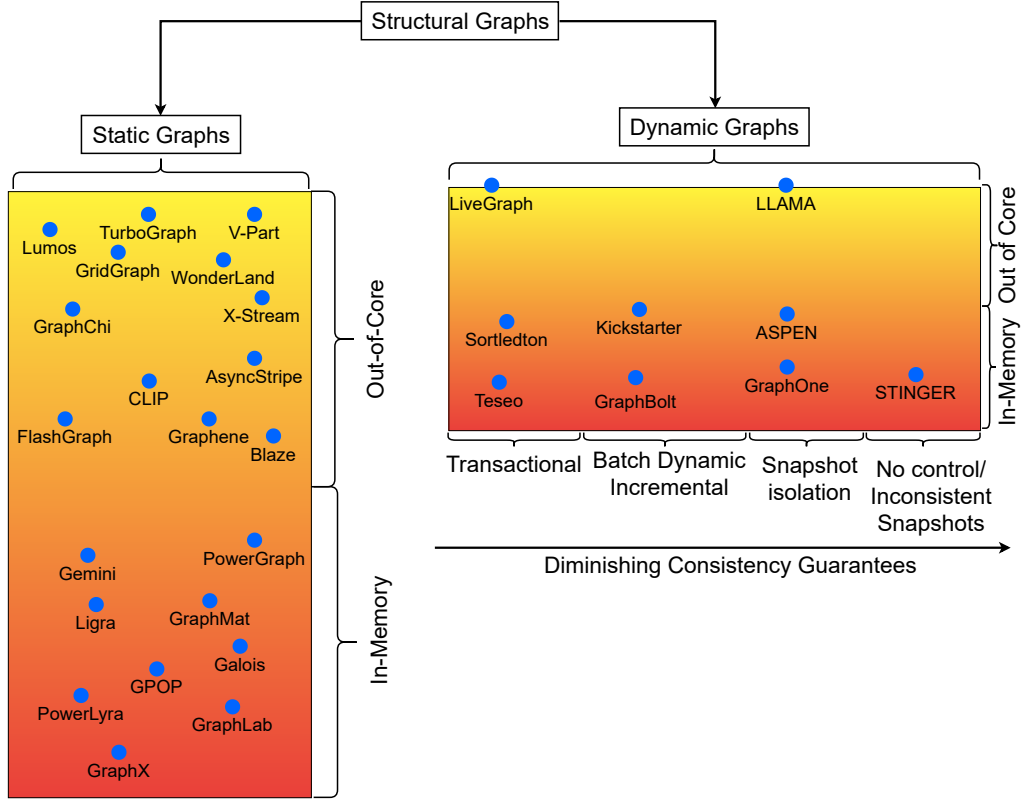


Figure 3.1: Design Space of Graph Data Management Systems

Graph processing systems are designed to execute iterative algorithms in distinct phases. A **Computation Model** defines these phases and the sequence in which they must be run. The user typically writes the algorithm in terms of these phases, and the system executes the algorithm by running these phases in sequence. The computing model is independent of the system’s scheduling model, which determines which vertices are selected for computation (i.e., remain active) in each phase.

The most well-known computation model is the **Bulk Synchronous Parallel (BSP)** model [90]. This model is based on the idea of supersteps, where the computation is divided into a sequence of supersteps, each consisting of a computation phase and a communication phase. The computation phase is where a user-defined function executes on each of the graph’s vertices, and the communication phase is where the vertices exchange messages. The BSP model *pushes* updates the vertices at the end of each superstep and applies these updates to the graph at the beginning of the next superstep. Examples of systems that use this model include Pregel [61] and Giraph [11].

The **Asynchronous** model is a computation model where the computation is not divided into supersteps. In the Asynchronous model, vertices make updates visible immediately; there is no clear delineation between supersteps. These systems need a consistency model for updating the edges and vertices. Examples of these systems include GRACE, ASPIRE, Giraph Unchained, CoRAL, and GraphChi [53].

Gather-Apply-Scatter (GAS) is a more general computation model that can describe both BSP and Asynchronous models. The GAS model consists of three phases: Gathering updates by *pulling* them from neighbors, computing and applying the updates, and *pushing* the updates to the neighbors. Systems built using this model

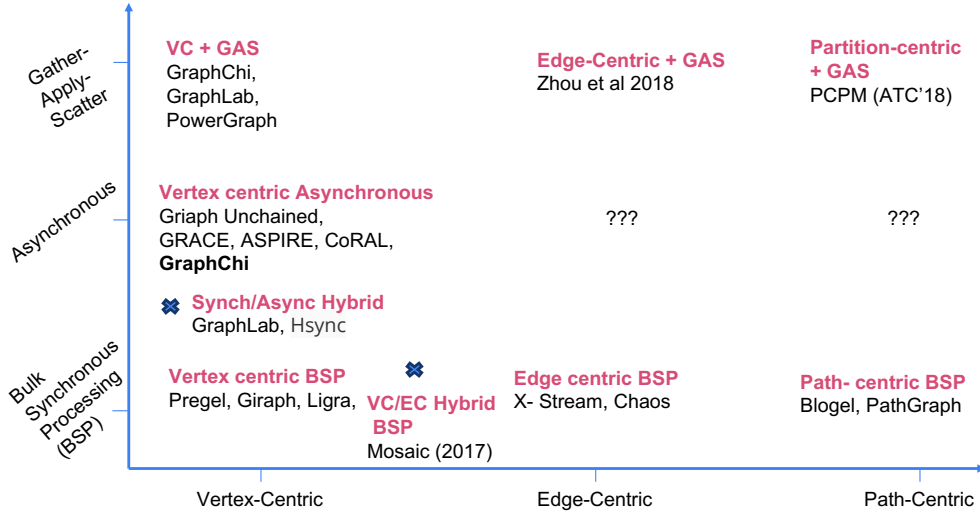


Figure 3.2: Splitting the space of static graph processing systems based on the programming model (x-axis) and the computation model (y-axis). The systems marked with a blue X are hybrid: Mosaic [58] is a hybrid vertex-centric/edge-centric system, while GraphLab [57] and Hsync [94] are hybrid in their computation model and switch between synchronous and asynchronous modes of computation.

include GraphLab [56], PowerGraph [44], GraphX [45].

The second axis to consider is the **Programming Model** used by the system. The programming model defines the interface provided to the user to write graph algorithms and, therefore, defines the unit of computation in a system. The most common programming model is the **Vertex-Centric** model, where the user writes a function that operates on each vertex of the graph. Each vertex program can read the state of the vertex and its neighbors and update the state of the vertex. The user provides a `compute()` function, a `get_value()` function to read the neighbors' state, and a `update_state()` function to update the state of the vertex.

The **Edge-Centric** model is another programming model where the user writes a function that operates on each edge of the graph. Edge-centric models propagate changes by updating the state of the edge and the state of the vertices connected by the edge. The user provides an `edge_scatter(e)` function that sends an update over the edge e , an `edge_gather(e)` function that reads the update from the edge e , and an `edge_gather(e)` function that applies the update to the edge's destination vertex $e.dest$.

Our last programming model is the **Partition Centric** model, where the user writes a function executed on each graph partition. This model is used by systems that are expressly designed to reduce cross-partition communication overhead in a distributed graph processing system.

The programming and computation models are orthogonal, and a system can use any combination. The design space of static graph processing systems can be split based on these two axes, as shown in Figure 3.2.

As the figure shows, the Vertex-Centric model is the most common programming model used by graph processing systems. There are some combinations of programming and computation models that are not used by any system. The more interesting thing to note here is that if the system provides a flexible API to interact with the graph, the user can implement any programming model on top of the system. Flexograph is designed to provide a flexible API that allows users to implement any programming model on top of it.

3.3 Dynamic Graph Processing Systems

Structural graphs are derived from property graphs, and since the property graph is dynamic, the structural graph must be updated as the property graph changes; this leads to the concept of dynamic structural graphs. The dynamic nature of these graphs requires some form of consistency. The structural graph must be updated in a manner consistent with the updates applied to the property graph while simultaneously ensuring that concurrent queries on the structural graph are not affected by the updates.

A dynamic system can apply updates to the structural graph directly without considering ongoing queries. When such a system applies an update (adding a new edge, for instance), it applies the update to the live graph data structure without creating any new versions. Any ongoing queries running on the graph data structure will see the update immediately, which can lead to incorrect or inconsistent results, query failures, or delayed convergence of iterative algorithms. STINGER [40] is an example of such a system that allows for fast updates to the graph data structure but does not provide any consistency guarantees.

A more sophisticated approach is to construct a new version of the structural graph as the updates arrive. Each update to the graph has an associated timestamp, and a system uses these timestamps to construct a versioned graph. The system can then provide a consistent view of the graph at a certain point in time by selecting the appropriate versions of the graph elements. Examples of systems that use this approach include GraphOne [52], LLAMA [59], and ASPEN [38].

Another approach is to batch updates and apply them together to maintain a single incremental view of the graph. The interesting challenge in these systems is to make sure that any in-progress computations are not affected by the updates and can continue to make progress with the new data. These systems are designed to use incremental algorithms that can minimize redundant computation by reusing results that have been computed before the graph structure mutates. GraphBolt [62], for instance, uses "dependency-driven incremental processing, in which it first tracks dependencies to capture how intermediate values are computed and then uses this information to incrementally propagate the impact of change across intermediate values". Kickstarter [92], GraphIn [79], and Tornado [81] are other systems that use a similar approach.

The most sophisticated approach to handling dynamic graphs is to provide a transactional interface to the user. Transactions help a system provide some or all of the ACID guarantees in the presence of updates. Good examples of transactional dynamic graph processing systems include LiveGraph [101], Sortledton [43], and Teseo [37]. LiveGraph stores the graph out-of-core by using a mmap-based storage engine and provides a transactional interface to the user. Sortledton is an in-memory system that provides a fully transactional adjacency-list based graph data structure. Teseo is also in-memory and transactional, but uses a "fat tree" (B+ tree derived) data structure to store the graph.

Clearly, the consistency model used by these systems falls along a spectrum (the left-right axis in the right branch of Figure 3.1 points in the direction of diminishing consistency guarantees). To construct a taxonomy, we use a simple binary feature to characterize this consistency axis: does the system support transactions or not? Graph Databases and a few systems (LiveGraph, Teseo, and Sortledton) support transactions which allow for fine-grained guarantees for individual updates to the graph; the remaining systems provide some form of batch-dynamism: isolation is coarse grained and therefore weaker.

3.4 Distillation of the Design Space

In this section, we provide a unified view of the design space that we discussed in the previous sections. So far we have identified that the design space of graph data management systems can be split based on the following classification features:

		Dynamic Graphs		Static Graphs
		Transactions	No Transactions	
Property Graphs	Graph DB		?	RO snapshots of Property Graphs?
Structural Graphs	Teseo Sortledton		GraphOne LLAMA Kickstarter GraphBolt STINGER ASPEN	Static GPS

?

This combination could exist, I don't know which system implements it

Figure 3.3: Distillation of the design space of graph data management systems. Please refer to Section §3.3 for an explanation of how the systems listed in the dynamic but not transactional category differ from one another.

1. Whether the system manages property graphs or structural graphs.
2. Whether the system handles updates to the graph (dynamic) or not (static)
3. Whether the system supports transactions for updating the graph

Based on these features, we can arrive at a grid that distills the design space of graph data management systems, as shown in Figure 3.3. The grid shows the different classes of graph data management systems based on these features. As can be seen in the grid, not all combinations of these features make sense to use together, for instance, a system that works only on static (read-only) graphs has no consistency requirement and therefore no need for transactions. As we saw in section §3.3, consistency becomes an issue only in the presence of updates. Therefore, the three axes flatten into the grid shown in Figure 3.3, since static (i.e., read-only) systems are agnostic with respect to transactions.

The main takeaway from this grid is that Graph Databases are the most general class of graph data management systems, because they are based on property graphs, structural graphs are merely a subset of these property graphs, and they support transactions to ensure consistency of updates.

All the other categories are specialized to handle a subset of the features provided by Graph Databases. Each of those systems work well for the specific use cases for which they are designed, but they are not as general as Graph Databases. Flexograph is designed to be a general-purpose graph data management system that can flexibly support all the specialized use cases depicted in this grid *without having to move the data* or sacrificing performance.

Chapter 4

Flexograph Architecture

We now introduce the Flexograph Graph Database architecture. Our goal in designing Flexograph is to achieve the performance of purpose-built graph-processing systems in the context of a transactional, dynamic, property-graph system. We developed Flexograph through iterative refinement, first investigating ways to get a production storage engine to mimic state-of-the-art in-memory structures. Next, we developed a programmatic API capable of implementing all the programming models cataloged in Section §3.2.1. Finally, we continued to refine the design to support the range of workloads described in Section §2.2.

If graph processing systems are those systems specialized for analyzing graph structure, then graph databases are the general-purpose systems designed to *store and manage* property graphs. Consequently, a graph database architecture must allow efficient storage and retrieval of both graph structural and properties while being able to support the broad range of graph workloads that we discussed in §2.2. Importantly, as computationally intensive analytics tasks operate on the structural graph, not the property graph, the challenge for graph databases is providing efficient analytic processing in the presence of the easier point queries and neighborhood queries that are common on property graphs. Therefore, we conclude that a graph database should store the graph structure independently from the properties and the labels associated with the nodes and edges. The layout optimizations needed by the structural graph can be different from those needed by the properties and labels, and this separation allows for more optimization opportunities.

Flexograph embraces this architectural decision to separate the graph structure from the properties and labels. Since providing good performance on analytics tasks is the more challenging the problems we address, development to date has focused primarily on this problem. While we have prototyped the rest of the property graph implementation, further design and implementation remains part of future work, which will be described in Chapter 6.

Although Flexograph does not yet handle properties, we have spent considerable time designing the system such that adding properties does not require duplicating the structure nor does it affect the performance of the analytics workloads we evaluate in Chapter 5.

The chapter breakdown of this thesis mirrors our decision to store the structure and properties of the graph independently. This chapter focuses on the structure of the graph, detailing our architecture for managing structural graphs. We build upon this architecture in Chapter 6, where we discuss our approach to storing properties.

For the rest of this chapter, we use the term graph to refer to a structural graph unless explicitly mentioned otherwise, and we use the term graph database to describe Flexograph, even though it does not yet handle property graphs.

The remainder of this chapter is organized as follows: we begin with a description of a common application to understand how the graph database is designed to make this application run efficiently. We then describe the

Flexograph architecture and the three components that make up the system. Following that, we describe how this architecture is implemented in the current prototype, discussing our storage engine, the design of our API, the graph representations we support, and the higher-level traversal primitives we support. We close with a description of our methodology for managing temporary graph properties and our design of an efficient bulk loader.

4.1 What should a Graph Database Provide?

In section §2.2, we discussed the different workloads that are commonly used on graph data and the access patterns they exhibit. We also discussed the data structures that are commonly used to support these workloads. A graph database, therefore should provide the following features:

- the ability to store the graph in **formats** that facilitate high-performance analytics by providing good performance on common access patterns
- a high-level **API** that abstracts the underlying data structures and provides a simple interface for the user to interact with the graph. The user does not need to know the internal representation of the graph to use and query her data.
- a **framework** to permit efficient implementation of common **operations**, such as neighborhood collection and traversal. Efficient iterators on the graph elements are critical for performance and should be first-class primitives in the system.

To illustrate how these requirements combine in the context of a real application, for this list of requirements, let us examine the following running use case: computing PageRank on a graph. The pseudocode for the PageRank algorithm is shown in Algorithm 1.

To implement the PageRank algorithm in a graph database requires accessing the following distinct parts of the graph:

- We need to access the vertex set to initialize the scores and the outgoing values (lines 5, 10 in).
- For each node, we need to access the incoming neighbors to compute the incoming total (line 12)
- As we process each vertex, we need to read and write (lines 15 and 16 respectively) its own score and then compute its outgoing value (line 18) .

For a graph database to support this workload efficiently, it must provide an API that allows the user to create a vertex iterator for efficient scans over the vertex set and a neighborhood iterator for efficiently scanning a vertex's neighborhood. This API is, of course, in addition to all the basic Create, Read, Update, Delete (CRUD) operations that are expected from a database system.

4.2 Architecture

With the requirements for a graph database in mind, we decompose the architecture of Flexograph into three components: efficient graph representations, a user-facing API for interacting with the graph, and a set of internal graph operators that serve as a glue between the two.

Algorithm 1 PageRank Algorithm as specified in GAP Benchmark [23]: this PR algorithm allows new values to be immediately visible (like Gauss-Seidel method). The highlighted lines indicate different accesses: access to the vertex set, access a vertex's neighborhood, and access the results vector. For a vertex v , $N^+(v)$ denotes its out-neighborhood, and $N^-(v)$ is the node's in-neighborhood.

Require: Graph $G = (V, E)$ where V is the set of nodes and E is the set of edges

Require: Damping factor $kDamp \in (0, 1)$, typically $kDamp = 0.85$

Require: Tolerance $\varepsilon > 0$, typically 10^{-4}

Ensure: PageRank vector Scores

```

1:  $init\_score = \frac{1.0}{|V|}$ 
2:  $base\_score = \frac{1.0 - kDamp}{|V|}$ 
3:  $Scores(v) = init\_score$  for all  $v \in V$ 
4:  $Outgoing\_vals(v) = 0$  for all  $v \in V$ 
5: for all  $v \in V$  do
6:    $Outgoing\_vals(v) \leftarrow \frac{init\_score}{|N^+(v)|}$ 
7: end for
8: for  $i = 1$  to  $max\_iters$  do
9:    $\Delta = 0$ 
10:  for all  $v \in V$  do
11:     $incoming\_total = 0$ 
12:    for all  $u \in N^-(v)$  do
13:       $incoming\_total \leftarrow incoming\_total + Outgoing\_vals(u)$ 
14:    end for
15:     $old\_score \leftarrow Scores(v)$ 
16:     $Scores(v) \leftarrow base\_score + kDamp \cdot incoming\_total$ 
17:     $\Delta \leftarrow \Delta + (Scores(v) - old\_score)$ 
18:     $Outgoing\_vals(v) \leftarrow \frac{Scores(v)}{|N^+(v)|}$ 
19:  end for
20:  if  $\Delta < \varepsilon$  then
21:    break
22:  end if
23: end for
24: return Scores

```

4.2.1 Graph API

The Graph API provides the user a means to interact with the graph that hides the underlying data structures or the storage backend being used to store the graph. This separation improves the portability of application code. The GraphAPI has three main components. It provides a **Structural API** for manipulating and querying the graph structure. This includes methods for adding and removing nodes and edges, getting a node's neighbors, and getting the in- or out-degrees of a node. The exact methods provided by the Structural API are listed in Table [Table 4.1](#). Iterator functions listed in the table are the methods that return a cursor to the graph elements: nodes, edges, in-neighbors, and out-neighbors; they can be used for efficient traversal of the graph.

The GraphAPI also provides a **Property and Label API** for manipulating and querying the properties and labels associated with the nodes and edges. This includes methods for adding and removing properties, getting the properties of a node or edge, and getting the labels associated with a node or edge. We will discuss this in more detail in [Chapter 6](#).

Not captured in the Structural or Property API are methods that allow the user to manage the graph database itself and query some metadata about the graph. We provide a set of **Management Functions** that allow the user to create a snapshot of the graph, close the graph database, and close all cursors. The **Metadata Functions** allow the user to query information about the graph, such as the maximum and minimum node IDs, and the number of nodes and edges in the graph, and to test whether the graph is weighted, directed, and/or labeled.

Different parts of the graph need different consistency guarantees. The structure of the graph can never be in an inconsistent state. For example, forward and reverse edges must be atomically created for an undirected graph, and an edge can only be created if its source and destinations nodes exist. Similarly, the properties and labels associated with the nodes and edges must be consistent with the structure of the graph; a property cannot be associated with a node that does not exist. However, the properties and labels can be updated independently of the structure of the graph. For example, the weight of an edge can be updated without affecting the structure of the graph.

Metadata in our system can always be inferred from the structure/properties of the graph, but it is often more efficient to store it separately. As such, while we provide methods to query and sync the metadata to persistent storage, we do not require it to have the same consistency guarantees as the structure of the graph.

Regardless of the individual guarantees of each operation, the API must have clearly defined semantics so that an application developer can reason about the behavior of the system. Additionally, the API must not make any assumptions about the application semantics. The API must return appropriate error codes to the application and let the application handle any errors that occur.

4.2.2 Internal Graph Operators for Defining Higher-level Access Primitives

The GraphAPI provides a user-facing interface for interacting with the graph. The internal operations that translate the intent expressed by the API into layout-specific operations are provided by the **Internal Graph Operators**. These operators are the glue between the API and the graph representations and leverage information about the physical layout to provide efficient implementations of key operations such as whole-graph traversal.

The implementations of these internal graph operators are backend-aware and can utilize any optimizations provided by the storage engine to improve the performance of the operations they implement. For example, the API [Table 4.1](#) provides basic primitives for getting the in- and out-neighbors of a node. However, using these primitives to implement a complicated traversal, such as PageRank, would be inefficient. The resultant code would need to make repeated calls to the API for each vertex, which is inefficient because the API calls can be expensive, and the user would have to manage the state of the traversal themselves. The code would be less tedious to write and maintain if the internal graph operators provided a framework for higher-level traversals, such as node, edge, and

Operation	Interface
Add node to graph	<code>void add_node(node n)</code>
Add edge to graph	<code>void add_edge(edge e)</code>
Check if node exists	<code>bool has_node(node_id id)</code>
Check if edge exists	<code>bool has_edge(node_id src, node_id dst)</code>
Get the edge between src and dst	<code>edge get_edge(node_id src, node_id dst)</code>
Get the node identified by the ID	<code>node get_node(node_id id)</code>
Get a random node from the graph	<code>node get_random_node()</code>
Delete the node with the given ID	<code>void delete_node(node_id id)</code>
Get in-degree for node with ID given	<code>node_id get_in_degree(node_id id)</code>
Get out-degree for node with ID given	<code>node_id get_out_degree(node_id id)</code>
Get a list of all nodes in the graph	<code>std::vector<node> get_nodes()</code>
Get a list of all edges in the graph	<code>std::vector<edge> get_edges()</code>
Get a list of in-neighbors for a node	<code>std::vector<node> get_in_nodes(node_id id)</code>
Get a list of out-neighbors for a node	<code>std::vector<node> get_out_nodes(node_id id)</code>
Get a list of in-edges for a node	<code>std::vector<edge> get_in_edges(node_id id)</code>
Get a list of out-edges for a node	<code>std::vector<edge> get_out_edges(node_id id)</code>
Get a list of IDs of in-neighbors for a node	<code>std::vector<node_id> get_in_node_ids(node_id id)</code>
Get a list of IDs of out-neighbors for a node	<code>std::vector<node_id> get_out_node_ids(node_id id)</code>
Delete the edge between src and dst	<code>delete_edge (node_id src, node_id dst)</code>
Update edge with new weight	<code>update_edge_weight (node_id src, node_id dst, weight w)</code>
Iterator Functions	
Get a cursor to the out-neighborhood	<code>OutCursor *get_outnbd_iter()</code>
Get a cursor to the in-neighborhood	<code>InCursor *get_innbd_iter()</code>
Get a cursor to the nodes	<code>NodeCursor *get_node_iter()</code>
Get a cursor to the edges	<code>NodeCursor *get_edge_iter()</code>
Metadata Functions	
Get the max node ID in the graph	<code>node_id get_max_node_id()</code>
Get the min node ID in the graph	<code>node_id get_min_node_id()</code>
Get the number of nodes in the Graph	<code>node_id get_num_nodes()</code>
Get the number of edges in the graph	<code>node_id get_num_edges()</code>
Is the graph directed?	<code>bool is_directed()</code>
Is the graph weighted?	<code>bool is_weighted()</code>
Sync metadata to the table	<code>void sync_metadata()</code>
Management Functions	
Create new snapshot of the graph	<code>void create_snapshot()</code>
Close the graph database	<code>void close()</code>
Close all cursors	<code>void close_all_cursors()</code>

Table 4.1: Graph API interfaces that are implemented by each representation in Flexograph. These interfaces handle the structure of the graph; interfaces for metadata and property updates are introduced in [Chapter 6](#).

neighborhood iterators. We discuss how Flexograph provides these iterators in [§4.3.4](#).

Moreover, the user might want to implement a traversal that accesses the graph in some specific fashion that cannot be expressed using the API. For instance, using a Hilbert curve to traverse the graph in a space-filling manner is a common technique to improve the locality of access in graph processing [64, 89]. This traversal is not expressible using the GraphAPI, and the user would have to implement it themselves. The ease of writing complicated custom traversals can significantly impact the usability of the system, and a graph database should provide a framework that allows the user to write custom higher-level traversals efficiently. We discuss our plans to extend Flexograph to provide motif matching and other custom traversals in [Chapter 7](#).

4.2.3 Graph Representations

We saw in [Section ??](#) that most structural graph processing systems use a CSR-like or Adjacency List-like representation to store the graph. A graph database should be able to persist these data structures to disk in a way that allows for efficient reconstruction and access. The database designer must identify how to store them in the storage engine and do so in a way that preserves the locality of access to vertices and their neighborhoods.

For storing a CSR, the database must store all incoming/outgoing edges for each vertex in a contiguous "chunk" and provide an index (or search) to identify the start and end of the chunk. To store an adjacency list, the neighborhood of each node must be materialized and stored in-line with the node; the nodes must be laid out sequentially in some ordering of their IDs. We describe how these representations are provided by Flexograph in [Section §4.3.3](#).

4.3 Implementation

Flexograph is a concrete realization of the architecture we described in the previous section. Our discussion of the implementation begins with a discussion of the storage engine we use to store the graph and how it's design influences ours. Following that, we describe how the GraphAPI described in [table Table 4.1](#) can be used to implement Page Rank ([Algorithm 1](#)) as discussed earlier. We then describe how we store the graph representations in our storage engine, how the API facilitates interaction with the data, and our iterator design for efficient traversal of the graph. We close with a discussion of how we manage temporary properties and our bulk loader.

4.3.1 Storage Engine

I don't think that's true.

A graph not fitting in RAM will take longer to access than a graph fitting in cache. WT will have to incur cache misses and run eviction.

If a read is not in cache, it will block. Be more precise.

Flexograph uses the WiredTiger (WT) [9] storage engine to store the graph. WT is a production-ready, high-performance, scalable, and extensible storage engine that is used by MongoDB as its storage engine. Our choice of WT is motivated by its excellent multithreaded read performance – [reads in WT are non-blocking](#). This is important for graph processing workloads, which are mostly read-heavy. [WT can handle data sets that are much larger than RAM without sacrificing performance](#), which is important for storing large graphs. It is ACID compliant and uses optimistic concurrency control to provide transactions – transactions in one thread do not block operations in other threads, but still provide snapshot isolation guarantees. This property gives us clear semantics for write-write conflicts to the same object in the database: all but one concurrent write to the same key will fail. An attractive feature that motivates our use of a key-value store is that it allows us to use different graph representations easily. The use of a KV store rather than a conventional database allows us to have persistent graphs while providing the control we need to experiment with graph-friendly layouts.

The use of WT has some implications for our implementation. WT is an **embedded** key-value store, which means the database is part of the application. WT databases, therefore, work on a single-process multithreaded model of parallelism. This means that the database is not shared across multiple processes, and there can be only one application with an open connection to the database at a time. The WT Application Programming Interface (API) requires a unique connection to the database for each process, and all the threads in that process can share the connection reference – the connection object (**WT_CONNECTION** object) is thread-safe. The transactional context of data operations in WT is managed by the session (**WT_SESSION**) object. The session object is not thread-safe, so each thread must create a private session object. The transactional context managed by the session object includes information about the transaction's isolation level, and more importantly, the position of cursor objects that are used to access the data. A cursor (**WT_CURSOR** object) is a handle to a position in the database, and it is used to find, read, and write data and traverse over records in the database. **WT_CURSOR** objects are not thread-safe, and each thread must create a private cursor object, but ~~they~~ can share the session object. The implication of these properties

~~they~~
cursors

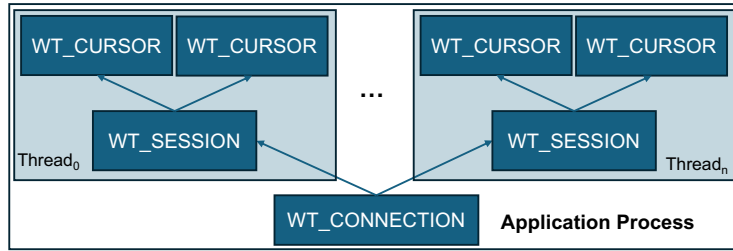


Figure 4.1: A typical application that uses WT resembles this diagram. The application has a single connection to the database, and each thread in the application has a private session object. Each thread can create private cursor objects using the session object to access the data.

of the single-process multithreaded model is that any application that uses WT resembles the application shown in [Figure 4.1](#).

Another interesting feature of WT is that it supports both **emphrow** stores and *column stores*. In a conventional relational database, each row in a table represents a specific entity, and each columns of the table stores an attribute of the entity represented by the row. In a row store, each entity is stored contigulously with all of its attributes; in a column store, all the values for a specific attribute are stored together, indexed by a row ID. WT supports a generalization of a column store, called *column groups*, in which multiple attributes can be stored together, indexed by the row-id, which is the WT primary key. An application describes the columns that comprise each column group when creating the table.

Traditional row-based storage is inefficient when we are primarily interested in only a small subset of properties. If we have a table with many columns, and we are interested in only a few of them, then we have to read the entire row from disk, even if we are only interested in a few columns. Using column groups in this case can significantly improve cache behavior and performance by reading only those WT files containing the columns needed by the query. This is especially critical in property graphs where nodes and edges can have an arbitrary number of properties. We will discuss how we use column groups to store properties in [Chapter 6](#).

4.3.2 Anatomy of a Flexograph Application

Consider a simple application that traverses the out-neighborhood of each node in the graph and calls a method `foo()` on each neighboring edge. The code sample for this application is shown in [Listing 4.1](#).

```

1  GraphEngine graphEngine(engine_opts);
2  graphEngine.calculate_thread_offsets();
3  #pragma omp parallel for
4  for (int i = 0; i < THREAD_NUM; i++)
5  {
6      GraphBase* graph = graphEngine.create_graph_handle();
7      adjlist found = 0;
8
9      OutCursor* out_cursor = graph->get_outnbd_iter();
10     out_cursor->set_key_range(graphEngine.get_key_range(i));
11     out_cursor->next(&found);
12     while (found.node_id != OUT_OF_BAND_ID)
13     {
14         for (node_id_t v : found.edgelist)
15         {
16             // Do something with the edge
17         }
18         out_cursor->next(&found);
19     }
20     graph->close();
21 }

```

Listing 4.1: Example application that uses the GraphAPI to find the out-neighborhood of a node. The highlighted parts of the snippet are described in Section §4.3.2

An application first creates a **GraphEngine** object (line 1). The **GraphEngine** class encapsulates the connection to the database. An application then calls the GraphEngine methods to create a graph handle (which encapsulates the session and associated cursors) and calculate the thread offsets (line 2). The thread offsets are used to divide the graph into equal parts for parallel processing.

A graph handle is a reference to an object of type **GraphBase** with which applications interact with the graph. The **GraphBase** object is created using the `create_graph_handle()` method of the **GraphEngine** object. The **GraphBase** object encapsulates the session and cursor objects and is the API end-point with which the user interacts. It provides all methods listed in Table 4.1, including the iterator methods.

The application can then use the **GraphBase**'s `get_outnbd_iter()` method to obtain an iterator to the outgoing edges of a vertex. In the example listed in Listing 4.1, the application creates an **OutCursor** object that can be used to traverse the out-neighborhood of a node. Iterators internally manage WiredTiger cursor operations and range checks, providing a simple interface for the user to traverse the graph: the user need only call the `next()` method to retrieve the next element in the iterator.

This simple example is instructive in understanding how the GraphAPI can be used to implement more complex traversals. A more complete example of a PageRank implementation using the GraphAPI is provided in the Appendix §9.1(Listing 9.1).

4.3.3 Graph Representations

Flexograph provides two graph representations: Adjacency Lists, and EdgeKey. EdgeKey representation is designed to be a CSR-like representation that can be stored in a key-value store. We had prototyped an RDBMS-like representation, but we found that it was not efficient for graph processing workloads. It was used as a reference benchmark

for early prototyping but has since been abandoned. We do not discuss it further in this thesis.

Flexograph uses unsigned integers¹ for storing the source and destination IDs. Flexograph uses 0 and `UINT32_MAX` to identify out-of-band values for cursor traversals, therefore these are reserved IDs not available to the user.

Node Table			Edge Table		
node_id	in degree	out degree	src_id	dst_id	e_weight
u	I	I	u	u	i

Out AdjList Table			In AdjList Table		
node_id	out_deg	out_nbd	node_id	in degree	in_nbd
u	I	u	u	I	u

(a) Adjacency List Representation. The "in_nbd" and "out_nbd" columns store the serialized byte array containing the neighbors of the node.

EdgeKey Table			
src_id	dst_id	val1	val2
u	u	I	I

(b) Original EdgeKey Representation

EdgeKey OutEdges Table				EdgeKey InEdges Table			
src_id	dst_id	val1	val2	dst_id	src_id	val1	val2
u	u	I	I	u	u	I	I

(c) Split EdgeKey representation with a materialized in-neighborhood table.

Metadata Table	
meta_key	meta_value
I	S

(d) Metadata table used by all representations

Figure 4.2: Graph Representations supported by Flexograph. Each representation also maintains a metadata table (d) to store the graph properties, such as maximum and minimum vertex ID, number of vertices and edges, and whether the graph is directed or weighted. `u` in this diagram denotes a serialized byte array, `I` denotes an unsigned integer for storing degrees, and `i` denotes a signed integer used for edge weights.

Adjacency List Representation

The primary storage for adjacency lists are a pair of tables whose primary keys are vertex IDs and whose data values are the inbound/outbound adjacency list. In addition, we maintain vertex and edge tables to store the properties and labels associated with each vertex/edge. [Figure 4.2a](#) illustrates this layout. The adjacency lists themselves, i.e., the `in_nbd` and `out_nbd` columns, are binary arrays containing the keys of the neighbors adjacent to the vertex on which the entry is keyed.

¹currently 4B unsigned ints, but this is a compile-time directive and can be changed to 8B ints

The Adjacency List representation does not require any secondary indexes. Although each edge insertion requires updates to at least three tables (Edge, Out AdjList, and InAdjList), they are reasonably efficient as they are all updates via primary key. If the DB is read-optimized, **the** the Node Table contains in and out degree counts, so an edge insertion or deletion requires two updates on the Node Table as well, again accessed via primary key. Adding a node requires only a single add to the Node Table, while deletion requires potentially many operations: one delete from the Node Table, one delete per neighbor from the Edge Table and at least one of the Adjacency List tables. Once again, if the database is read-optimized, once again, we need additional updates to the Node Table for each neighbor.

EdgeKey Representation

The EdgeKey representation is designed to approximate CSR inside a key-value store. Our original EdgeKey representation [Figure 4.2b](#) used a single table to store both nodes and edges. It exploits the fact that WiredTiger stores keys in lexicographically sorted order. This means that if we have a composite key (a, b) , such that $a, b > 0$, and we want to scan all keys with $(a, *)$, we can just position the cursor at $(a, 0)$ and then iterate over all keys $(a, *)$ by sequentially moving the cursor. [Figure 4.2b](#) shows the original EdgeKey Table. `src_id` and `dst_id` form a composite primary key of the table. We encode the type of object represented in the primary key: entries of the form $(x, 0)$ represent nodes and those of the form (x, y) represent edges. The value associated with a key are two unsigned ints, whose meanings vary by the entry type:

- When the entry represents a node and the table is created to be *read optimized*, values are the `in_degree` and `out_degree` respectively. The default, however, is to use a *write optimized* layout, and these values are **uninterpreted fields**.
- For an edge, the first entry in the array contains the edge weight, if the graph is weighted. For an unweighted graph, `val1` is set to the out-of-band value. `val2` is always set to the out-of-band value for edges.

There is only one index created on the edge table: (dst_id, src_id) . This index is used for iteration over the in-neighborhood of a node.

During benchmarking, we realized that the EdgeKey representation performs *horribly* on random in-neighborhood seek-and-scan workloads (see [Chapter 5](#)), because the adjacency lists contain only outneighbors. That is, finding the in-neighbors of vertex x required either scanning the entire database to find vertices whose adjacency lists contained v or maintaining and using a secondary index on in-edges. Both solutions were unacceptably slow.

To remedy this, we introduce a new representation that materializes the in-neighborhood of a node. We call this representation a *Split EdgeKey* representation, and it is shown in [Figure 4.2c](#). The Split EdgeKey representation and the basic EdgeKey representation present a classic time-space trade off: the split EdgeKey representation requires more disk space but makes it significantly faster to access a node's in-neighborhood.

4.3.4 Internal Graph Operators

The internal operators provide the glue between the graph API and the key-value store. A KV store provides functionality designed for individual item access, but graph elements combine to form higher-level abstractions, such as neighborhoods. The internal graph operators implement these abstractions.

We discussed the API provided by Flexograph in [Table 4.1](#); we now discuss the more interesting internal graph operators that are used to provide iterators and transient property storage.

Iterators

We examined the need for iterators in §4.2.2 and saw how an application might benefit from a higher-level abstraction for graph traversals than the basic cursor operations provided by WiredTiger in section §4.3.2. The iterators provided by Flexograph are designed to be efficient and easy to use.

Analytics tasks on graphs require finding the in- and out-neighborhoods of some or all nodes and traversing through them to perform the required computation. Finding a node in a WiredTiger database is essentially a B-Tree lookup, and requires indirection to locate the correct starting position of the node’s neighborhood in the B-Tree leaf node. Once the starting position has been found in the tree, traversing the neighborhood can be accomplished by moving through the entries in the leaf node until the invariant (matching on the source or destination vertex ID, for instance) stops holding.

There are methods provided by WiredTiger for doing this: the cursor must be positioned using the `set_key()` followed by a call to `search()` for that key. If found, the cursor can be moved using calls to `next()` and `prev()` methods. While these methods are useful, it is not convenient to use them as such. First, the cursors must be associated with a key range that it must traverse within. The application must repeatedly check to see if the found key is in range for each call to `next()`, and this can quickly overwhelm the core logic. Second, since the API is designed to be used by applications, it is more convenient to have a higher-level abstraction that can be used to traverse the graph.

We provide Iterators as a higher-level abstraction to plain cursors for graph traversals. The basic interface for an iterator is shown in Figure 4.3 and Listing 4.2.

```

1  class table_iterator
2  {
3  protected:
4      WT_CURSOR *cursor = nullptr;
5      WT_SESSION *session = nullptr;
6      bool is_first = true;
7      bool has_next = true;
8
9  public:
10     table_iterator() = default;
11     virtual ~table_iterator() = default;
12     bool has_more() const { return has_next; };
13     virtual void reset()
14     {
15         cursor->reset(cursor);
16         is_first = true;
17         has_next = true;
18     }
19     void close() { cursor->close(cursor); }
20 };

```

Listing 4.2: Table Iterator base class

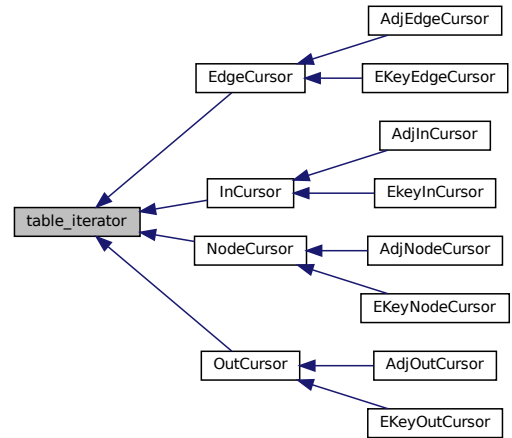


Figure 4.3: Inheritance diagram for the iterator classes

All iterators inherit from the base class `table_iterator` and must implement additional methods to set the key range, handle calls to `next()`, and handle the cursor going out of bounds. The inheritance diagram for the iterator classes is shown in Figure 4.3. `InCursor`, `OutCursor`, `NodeCursor`, and `EdgeCursor` are the iterator classes that are used to traverse the in-neighborhood, out-neighborhood, nodes, and edges respectively. Each

representation provides an implementation of these iterator classes. The GraphAPI provides iterator functions (listed in [Table 4.1](#)) that return a base class pointer (**InCursor***, **OutCursor***, **NodeCursor***, or **EdgeCursor***). These base class pointers are used to traverse the graph.

An application might need to position the cursor using a partial key. For example, an application might want to find all edges that have a source ID of 5. Since the WiredTiger cursor requires a complete key to be set, the application must construct a key that has the source ID set to 5 and the destination ID set to the out-of-band value (0 in our case). The iterator code must then use the WiredTiger `search_near()` method to position the cursor that is nearest to the key provided; the position of the cursor is then adjusted to the correct key. This is a common operation and is encapsulated in the `set_key()` method of the iterator classes and is used by the application to position the cursor for a scan. [Chapter 5](#) details a seek-and-scan graph microbenchmark to show the performance of the iterator classes and compare our performance to performing similar operations on other in-memory data structures.

Managing Transient Properties

There exist analytics tasks, e.g., PageRank, that compute an important property using an iterative algorithm, such as that shown in [Algorithm 1](#). When the computation completes, the computed value should be stored persistently as a property in the database. However, while the value is being computed, repeatedly storing the intermediate values to the property graph persistent storage is exorbitantly expensive. We call such values *Transient Properties*. Transient properties are usually maintained in memory and are freed when the application is closed. We provide a mechanism for the application to **persist the transient properties as they are being computed** and to reconcile them with the graph structure between two runs of the application.

The transient attributes are stored in a file-backed `mmap` region. An application can request a `mmap` region of a certain size from the Flexograph GraphAPI. The GraphAPI will return to the application a pointer to the region, and the application can use this pointer to store the transient attributes, just as it would in memory. The GraphAPI handles the `msync` and `munmap` system calls to ensure that the data is persisted to disk on closing the database connection or when the application requests it. Additionally, the GraphAPI provides a method for the application to find and map the last attribute region that was persisted to disk.

Between two runs of the analytics application, the graph structure might have changed. To use the old transient attributes, the application must reconcile the altered graph structure with the persisted attributes it stored from the last run. The GraphAPI provides a `reconcile()` method to do this. **The `reconcile()` method makes a few assumptions about the application's use of the transient attributes:**

- The application does not reuse the vertex ID of a deleted node. An application that does this must ensure that the transient attributes are updated to reflect the new node ID.
- The schema of the transient attributes has not changed between the old and new graph states. The `structs` used to store the transient attributes must be the same between the old and new graph states.
- The application knows apriori the number of transient attributes (and therefore the size of the `mmap` region) that it needs to store. The `reconcile()` method does not handle dynamic resizing the `mmap` region.

With these assumptions, the `reconcile()` method first opens the file containing the old transient attributes and `ftruncate()`s it to the size of the new transient attributes. Then the file descriptor to this resized file is `mmap`ed into memory and returned to the application as a pointer.

This is a basic mechanism for persisting transient attributes, and it can be extended to provide more functionality as needed.

4.3.5 Bulk Loader

Bulk loading is the process of inserting a large number of records into the database, either from a file or from another database. Constraint checks, index updates, and logging are some of the operations that are disabled during bulk loading to speed up the process.

Graph databases, in particular, must support bulk loading as the graph structure can be quite large and the time taken to insert the graph elements can be quite long. The interconnected nature of the graph means that the graphs might need to be inserted in a particular order: Neo4j's bulk loader requires nodes to be inserted before edges and that there not be any duplicate entries [4].

Flexograph also provides a bulk loader that can be used to insert a graph into the database. We provide two versions of the bulk loader: one that uses the standard transactional insertions and another, more optimized, version that is built using the WiredTiger bulk loading infrastructure [8].

WiredTiger provides an option to bulk load data into a *newly created* object. The bulk loader uses a special cursor that is opened with a `bulk` configuration parameter passed to it during creation. An object that is being bulk loaded is not accessible from any other cursor, and there can only be one bulk load cursor open on an object. This means that bulk loading data into a table can not be multithreaded. Additionally, the bulk load cursor expects keys to be inserted in sorted order; failure to do so results in an error. The expected sort order is achieved by using `sort` utility [3] to pre-sort the data.

The bulk loader provided by Flexograph is designed to insert large graphs into the database as quickly as possible while using all available cores. We detail the design of the preprocessing pipeline needed to sort the data and the two bulk loader designs.

4.3.6 Preprocessing Graphs for Bulk Loading

Since the bulk loader expects pre-sorted keys, we must first sort the graph. We use the `UNIX sort` [3] utility to sort the data including the options to *numerically* sort the graph by source vertex ID and destination vertex IDs and the `-parallel` option to perform the sort in parallel.

Following the sort, we split the edge list into multiple files, each containing an approximately equal number of edges. Since we want maximum parallelism, we use the number of cores available to determine the number of files, if the user does not present an explicit value.

We compute the number of nodes by replacing the delimiter (usually a tab or a comma) with a newline and then using the `uniq` and `wc` utilities to count the number of unique nodes.

Since our Adjacency List and EdgeKey representations both require the reverse edges (in-neighborhood for a node), we reverse the edges in the edge list, sort them, and split them into multiple files. The number of files created for the reverse edges is the same as the number of files created for the forward edges.

Once the edges have been sorted and split, we construct adjacency lists for the nodes. Each thread reads a shard of the edge list and begins to construct an adjacency list. Since the files are sorted, the adjacency list for a node can be constructed by reading the file sequentially and adding the destination vertex ID to the adjacency list. Once a new source ID is encountered, the adjacency list for the current node is considered complete and is written to the adjacency list file. The thread continues doing this until the end of the file is reached.

At the end of this multithreaded adjacency list construction, we need to make sure that we don't have adjacency lists split across two files. To do this, we read the last adjacency list (last line) from the i th file and the first adjacency list (first line) from the $i + 1$ th file, and, if appropriate it, merge them, delete the last line from the i th file and write the merged adjacency list to the first line of $i + 1$ th file.

At the end of these preprocessing steps, the graph has been converted to an adjacency list format, and the files

are ready to be bulk-loaded into the database.

4.3.7 Transactional Bulk Loader

The transactional bulk loader is a simple bulk loader that uses the WiredTiger cursor methods to insert nodes and edges into the database. Since we need this bulk loader to be multithreaded, we use a separate session for each thread and a separate cursor for each session. Since we need to open multiple cursors to the same database object, we do *not* provide the `bulk` configuration parameter during the cursor creation. The transactional bulk loader is slower than the bulk loader that uses the WiredTiger bulk loading infrastructure since it uses implicit transactions to insert the data.

The transactional bulk loader is used to insert data into all tables needed by the graph representation and insert the adjacency lists into the database. Each thread is passed a reference to the **WT_CONNECTION** object and establishes a session and cursors to the appropriate tables. The thread then reads a shard of the adjacency list file line by line and inserts the adjacency list into the database. This is repeated to insert the nodes and edges into the database. The cleanup step in the bulk loader is to insert metadata about the graph into the metadata table before closing the connection.

4.3.8 Modified WiredTiger Bulk Loader

The transactional bulk loader works fine for small graphs, but for large graphs, the bulk loader that uses the WiredTiger bulk loading infrastructure is faster. WiredTiger provides a bulk loading infrastructure that can be used to bulk load data into a *newly created* object, but as we have discussed, does not allow multiple threads to bulk load data into the same object. Since bulk cursor inserts are single-threaded, they don't need auto-commit transactions because the updates are not visible until after the bulk cursor is closed.

To remove this single-threaded constraint, we modify the bulk loader to use multiple threads to bulk load data into the database². The basic idea is to create leaf pages in the B-Tree and alter the reconciliation³ logic to create the internal nodes of the B-Tree. Since no internal pages are generated as the bulk loader does a pass over the data, the bulk loader does not need to hold any internal page locks and can be multithreaded.

The modified bulk loader is used to insert the adjacency lists into the database and uses the same preprocessing pipeline as the transactional bulk loader. Each thread uses a separate session and cursor to bulk load its shard of the adjacency list file.

The bulk loader has been extended to create other tables needed by the graph representation and insert the nodes and edges into the database.

The modifications needed for this bulk loader are based on the WiredTiger 3.2.1 release, and the bulk loader is not part of the main WiredTiger distribution. The rest of the project uses the WiredTiger 11.0.0 release, but since the data layouts between the 3.2.1 release and the 11.0.0 release are the same, the bulk loader can be used with the 11.0.0 release. We are working on porting the bulk loader changes to the 11.0.0 release of WiredTiger.

²I did not write this specialized bulk loader. Keith Bostic very generously agreed to do this for us. The bulk-loader changes can be found at the `graph-database-bulk-load` branch on the WiredTiger repository. The driver to use this bulk-loader is part of the Flexograph codebase

³Reconciliation refers to the process of converting the in-memory version of a page to a more compact form for file storage

Chapter 5

Evaluation

This chapter details the evaluation strategy that we are going to follow to assess the performance of Flexograph and the results that we have obtained. We start by presenting the research questions that our evaluation answers. We then describe the datasets and benchmarks that we use for the evaluation and the experimental setup. Following that, we present our results to date and the plans to further evaluate the system. Finally, we conclude the chapter by summarizing the key findings of the evaluation.

5.1 Research Questions

So far we have presented the design and implementation with a special focus on making it competitive with the state-of-the-art graph analytics systems. The objective has been to create a graph database that is as general as any standard graph database, but also as fast as any specialized graph analytics system.

In this chapter, we evaluate the performance of Flexograph on structural graphs to answer the following research questions:

1. How do the **EdgeKey and Adjacency List representations compare in terms of performance and memory usage**? We want to understand the trade-offs between the two representations and how they affect the performance of graph processing. We evaluate this using microbenchmarks that focus on the performance of basic graph operations such as edge insertion, edge deletion, and edge lookup.
2. How much time do competitor systems spend in preprocessing the graphs? **Including preprocessing time, does Flexograph perform better?** What is a good metric to measure the "preprocessing time" in Flexograph considering that we have no explicit preprocessing step?
3. How does Flexograph perform compared to other **in-memory graph processing systems** on standard graph analytics workloads? This corresponds to the case where the graph is small enough to fit in the WiredTiger cache and the graph processing is done in memory.
4. How does Flexograph perform compared to other **out-of-core graph processing systems** on standard graph database workloads? This corresponds to the case where the graph is too large to fit in memory and some evictions from the WiredTiger cache are necessary.
5. How does Flexograph perform compared to other **dynamic graph processing systems** on specialized graph analytics workloads? This corresponds to the case where the graph is updated frequently, and the graph

processing is done in memory. We will assess the number of transactions that Flexograph can handle per second. We will also evaluate the strong scaling of Flexograph by increasing the number of threads and measuring the throughput.

6. How does the performance of Flexograph change with **different graph sizes and types**? We know that the performance of graph processing systems is highly dependent on the graph structure. We want to understand how Flexograph performs on different types of graphs (e.g., social networks, web graphs, uniform random graphs, and low-degree high-diameter graphs such as road networks).
7. Can we use Flexograph's GraphAPI to write **edge-centric and vertex-centric algorithms**? Does one representation perform better than the other for these algorithms? We are not interested in measuring the performance of these algorithms, but rather in understanding how the GraphAPI can be used to implement them¹.

5.2 Datasets and Benchmarks

A few graph analytics algorithms are commonly used in the graph processing community. The most common of the graph algorithm "kernels" are formally specified and have a performant and *correct* implementation provided by the GAP Benchmark Suite (GAPBS) [24]. More importantly, though, using this reference implementation allows us to draw a more direct comparison between the performance of our graph database and other graph processing systems. When running fair comparative benchmarks, it is important to execute the same algorithm implementation on all systems. Every graph processing system has its own native implementation of graph kernels. Comparing these different implementations can lead to misleading conclusions, because it is challenging to determine whether the observed performance is due to the inherent efficiency of the system or due to the use of a more efficient kernel implementation.

We use PageRank(PR), Breadth-First Search(BFS), Connected Components(CC), Single-Source Shortest Path (SSSP), and Triangle Counting(TC) algorithms from GAPBS to evaluate the performance of Flexograph. These algorithms are widely used in the graph processing community [65] and are representative of the different types of graph algorithms that are used in practice.

5.2.1 Modifications to GAPBS

While the graph API used in GAPBS is standard and available in Flexograph, there are a few modifications we had to make to get GAPBS working, which we describe in Appendix §9.1.

Flexograph ingests data faithfully; we never change node or edge IDs, even though some systems (Teseo [37], Stinger [40], GraphOne [52], LiveGraph [101]) map user provided vertex IDs to a logical ID used by the system. We always use user-provided node IDs and edge IDs to store and access the graph. This is a design decision that we made to simplify the graph representation and to avoid the overhead of maintaining a mapping between the user-provided IDs and the internal IDs. This design decision has only a slight implication for our use of GAPBS implementation, which assumes that the nodes are numbered from 0 to `MAX_NODE_ID`. Instead of allocating our vector and queue classes (used to store the results) with a size equal to the number of nodes, we instead initialize the classes with a buffer large enough to hold `MAX_NODE_ID` number of nodes in the graph. This is a minor change and does not affect the performance of the GAPBS implementation, especially considering our use of transient attributes.

¹Margo's dancing elephant analogy: "It's not about how well the bear dances, but that it dances at all."

Graphs	Vertices $ V $	Edges $ E $	Uncompressed Size
cit-Patents [16]	3.77M	16.50M	268 MB
dimacs9-USA [2]	23.94M	57.70M	940 MB
twitter_mpi [33]	52.57M	1.96B	37 GB
kronecker-s26, uniform-rand-s26	67.10M	1.07B	18 GB
kronecker-s28, uniform-rand-s28	268.43M	4.29B	74 GB
kronecker-s30, uniform-rand-s30	1.07B	17.18B	

Table 5.1: The graphs used for the evaluation with their number of vertices and edges, and the size of the uncompressed dataset.

5.2.2 Datasets

We use the datasets summarized in Table 5.1 for our evaluation. The selection of the dataset is driven by the need to evaluate the performance of Flexograph on different types of graphs. We use the **dimacs9-USA** dataset [2], which is a road network graph of the USA. It is interesting because road network graphs are low-degree, high-diameter graphs and are known to be challenging for graph processing systems. We use the **twitter_mpi** dataset [33], which is a social network graph. Social network graphs are known to have a power-law degree distribution and are predominantly used in graph processing system benchmarks. We use the **kronecker** and **uniform-rand** datasets, which are synthetic graphs generated using the Kronecker graph generator. These graphs are used to evaluate the performance of graph processing systems on graphs with controllably increasing sizes: from scale factor 26 (67.10M vertices, 1.07B edges) to scale factor 30 (1.07B vertices, 17.18B edges). We use the Smooth Kronecker graph generator [19] to generate graphs with a power-law degree distribution, while the GAPBS uniform-rand graph generator is used to generate graphs with a uniform degree distribution.

Remind me to tell you about
cache sizing in WiredTiger
as well as eviction thread number sizing

5.3 Experimental Setup

We conducted all experiments on a single socket workstation equipped with an Intel(R) Xeon(R) W-2275 CPU @ 3.30GHz processor. The workstation has 14 cores and 28 threads, 128GB of RAM, and a 1TB NVMe SSD. We do not disable hyperthreading on the machine, and we use the default settings for the Linux kernel. The source code was compiled with GCC 11.4.0 (which ships default on Ubuntu 22.04 LTS release) with the $-O3$ optimization flag. We use the WiredTiger v11.0.0 for the Flexograph but set the WiredTiger cache size to 10GB. This cache size is large enough for small graphs but small enough that the largest graphs cannot fit in memory.

5.4 Systems for Comparison

Flexograph is a graph database that can be used in specialized contexts for structural graph analytics, dynamic structural graph processing, and (most broadly) graph database workloads. We span the whole grid of graph data management systems shown in Figure 3.3 and must therefore compare Flexograph to all classes of graph processing systems.

In particular, we compare Flexograph to the systems mentioned in Table 5.2. The Grafaree project mentioned

	Query types?	Benchmarks?			
System Type	Benchmarks	How?	Relevant Flexograph Code Ready?	Comparison Done?	Competitors
OOO Analytics	GAPBS kernels	GAPBS	DONE	TBD	GraphChi [53], X-Stream [75], Lumos [91], Blaze [51]
In-Memory Analytics	GAPBS kernels	GAPBS	DONE	IN-PROGRESS	GAPBS [23], Ligra [82]
Dynamic graph stores	Analytic Benchmarks, Transactional workloads	GAPBS + Grafaree	IN-PROGRESS	TBD	GraphOne, Teseo, LiveGraph
Graph Databases	LDBC queries, Analytic queries	GraphBenchmark [55]	TBD	TBD	Neo4j [13]/Arango [10] (single-node)

Table 5.2: Systems to which we compare to evaluate the performance of Flexograph on **structural graphs**.

The systems are divided into four categories: Out-of-core(OOC) Analytics, In-memory Analytics, Dynamic graph stores, and Graph Databases.

in Table 5.2 is an in-house dynamic graph processing system benchmark framework that has been designed by Milad Rezaei, a former M.Sc. student at UBC. It uses a custom read-write workload generator to evaluate the performance of dynamic graph processing systems under varying degrees of randomness of access. Like Teseo, Grafaree also uses GAPBS kernels to evaluate the analytics performance of dynamic graph processing systems. We plan to use Grafaree to evaluate the performance of Flexograph on dynamic graph processing workloads, and there is still some work to be done to get the Flexograph driver ready for Grafaree (the timeline for this is discussed in Chapter 8).

5.5 Microbenchmarks

We plan to use microbenchmarks to evaluate the performance of the EdgeKey and Adjacency List representations and the overheads they introduce relative to a standard CSR representation.

The microbenchmarks cover the following operations:

1. **Edge Insertion/Deletion:** We measure the time taken to insert or delete a single edge in the graph. We will also measure the time taken to insert/delete a batch of edges into the graph. This microbenchmark leverages the Grafaree workload generator to generate random edge insertions and deletions.
2. **Vertex Insertion/Deletion:** We measure the time taken to insert or delete a single vertex in the graph. We also measure the time taken to insert/delete a batch of vertices into the graph. This microbenchmark leverages the Grafaree workload generator to generate random vertex insertions and deletions.
3. **Vertex Seek:** This measures the time to find the vertex in the graph representation. We measure the time taken to find a vertex by its ID. We compare this to the time taken to find a vertex by its index in the CSR representation.
4. **Neighborhood Scan:** This microbenchmark evaluates the time taken to scan the neighborhood of a randomly chosen vertex in the graph. We measure the time taken to scan the neighborhood and normalize it by the number of edges in the neighborhood, getting the average time per edge. We compare this to the time taken to scan the neighborhood in the CSR representation.

The microbenchmarks should give us a good idea of the overhead of using the GraphAPI for accessing an in-memory graph compared to a native in-memory CSR, which allows us to determine the overhead introduced by the use of GraphAPI and WiredTiger.

What is pre-processing for GAPBS?

You need to tell us how
GAPBS implementation
works.

What WT edge representation do you use? A? B? C?

tell us
here the
data size,
so we know
if it's OOC

You can
make this
break across
lines

		Flexograph	GAPBS	GraphChi
PageRank	Algo time	0.481	0.063	2.700
	Preprocessing Time	1.2	2.250	1.461
	Total Time	1.681	2.313	4.161
Triangle Counting (exact)	Algo time	Trust Triangle: 3.770 Cycle Triangles: 0.067	NA	2.448
	Preprocessing Time	1.2	NA	6.172
	Total Time	Trust Triangle: 4.970 Cycle Triangles: 1.267	NA	14.792

Table 5.3: Preliminary benchmark performance data using the PageRank and Triangle Counting algorithms on the cit-Patents graph. The times are reported in seconds. The preprocessing time mentioned for Flexograph is the time it takes to open the connection and warm the cache.

5.6 Preliminary Results

So far, the evaluation has focussed on ensuring that the Flexograph project is competitive with the state-of-the-art, in-memory graph processing benchmark: GAPBS. The idea here is to ensure that using Flexograph as the graph storage layer does not introduce a significant slowdown in the performance of the *same* benchmark algorithm when run on an in-memory CSR representation. Another important aspect of the evaluation is to demonstrate competitive performance with other out-of-core graph processing systems. We compared the performance of Flexograph with GraphChi [53] on a subset of the benchmarks.

How do you warm the cache? Don't you
need to begin processing to warm it?

Table 5.3 shows the preliminary performance data that we have obtained so far. The preprocessing time for Flexograph is the time taken to open a connection to the graph database, create graph handles, and warm the cache. Comparing the algorithm run time, Flexograph is slower than GAPBS, but including the preprocessing time for both, we are faster than GAPBS. GAPBS is a highly optimized in-memory graph processing system, and it is not surprising that it outperforms Flexograph.

However, GAPBS does not provide an exact triangle counting implementation, and we had to use our own Trust Triangle and Cycle Triangle implementations, which are comparable with the algorithms used in GraphChi (Cycle triangles). We significantly outperform GraphChi on this benchmark, which is a good sign for the performance of Flexograph on structural graph analytics workloads.

You need more analysis here. Profiling, details of where the time goes. If you just tell us 'hey, I'm better than GraphChi', how do I know I can believe you?

Chapter 6

Storing Property Graphs

We now discuss our future work to store property graphs in Flexograph and the design decisions that we need to make. There are two proposals for storing property and label information – inline with the element or separately using column stores. We provide mechanisms to do both and perform a comparative study of how they perform under a variety of workloads.

6.1 Example Property Graph Schema

We use the LDBC Financial Benchmark [71] schema to aid our discussion of property and label storage in Flexograph. As shown in Figure 6.1, the schema consists of 5 vertex types and 9 edge types. Each vertex type has a set of properties, and each edge type has a set of properties.

Translating this schema into a Flexograph compatible schema, we have to make the following decisions:

1. How should we store **vertex properties** in our representations? Should we store them inline with the vertex or separately using column stores?
2. How should we store **edge properties** in our representations? This too can be stored inline with the edge or separately using column stores.
3. How should we handle the case of **non-unique vertex IDs across different vertex types**? Should we use a single vertex ID space or separate vertex ID spaces for each vertex type to ensure uniqueness?
4. How should we handle the case where **edges of different types have exactly the same source and destination vertices**? Should we use a single edge ID space or separate edge ID spaces for each edge type to ensure uniqueness? For example, in the LDBC Financial Benchmark, the edge types `transfer` and `withdraw` connect the same vertex types `account` and `person`. We must ensure that we can distinguish different edges connecting the same nodes `transfer` and `withdraw`.
5. How should we index the graph to quickly access vertices and edges based on their labels and properties? There has been prior work in this space [47] to identify how to best use columnar storage for graph data. We will build on this work to create an index that can be used with any property graph schema.
6. How can we make the index creation schema-agnostic to be used with any property graph schema?

We have identified strategies for items 1 - 4 and will discuss them in the following sections. Indexing the graph based on labels and properties is a future work item that we will discuss in Chapter 7.

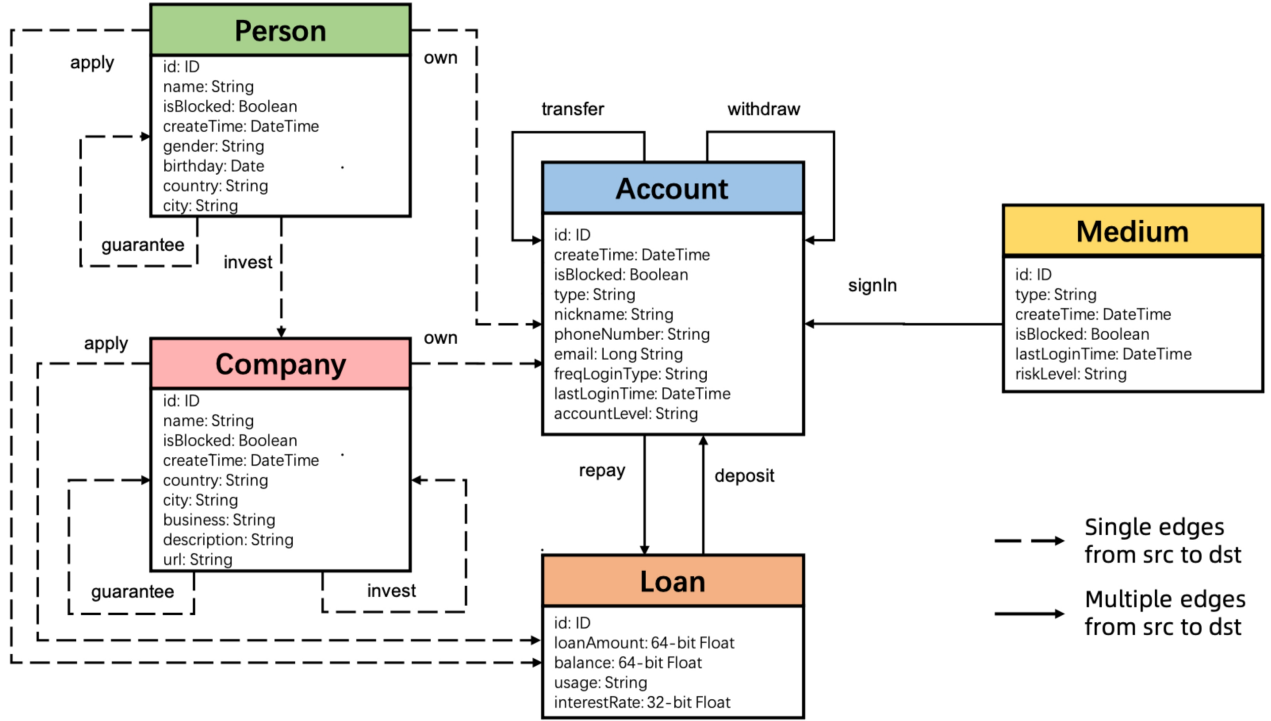


Figure 6.1: The data schema used in the LDBC Financial Benchmark [71]. The schema consists of 5 vertex types and 9 edge types. Each vertex type has a set of properties, and each edge type has a set of properties. The schema is a property graph schema that defines the types of vertices and edges and the properties associated with them. The figure has been grabbed from [71, Figure 3.1]

6.2 Additional API calls for property and label management

In addition to the existing API calls for graph traversal shown in Table 4.1, we need to add API calls to manage properties and labels. We propose the API calls in Table 6.1 to manage properties and labels in the graph.

This API will allow users to manage properties and labels in the graph. These API calls will have similar semantics to the existing API calls for graph traversal and follow the same design principles we outlined in §4.2.1.

6.3 Property Storage: Embedded vs. Split

The design(s) proposed here were derived from extensive group discussions with Jack Gong about storing labels and properties in Flexograph for his thesis. He described the inline (embedded) property representation in his thesis, and we’ve borrowed that prose here. My work will implement both approaches to determine the conditions under which each approach is preferred.

We can store properties in two ways: embedded with the vertex or edge or separately using column stores. We hypothesize that the embedded approach is more efficient for small graphs or graphs with few properties. On the other hand, the split approach is likely more efficient for large graphs or graphs with many properties. We will implement both approaches in Flexograph and compare their performance under the LDBC Financial Benchmark

Metadata Operations	
Create/update/delete a label	<code>void create_label (string label_key, string label_value)</code>
Get label from a name	<code>string get_label (string label_key)</code>
Create/update/delete a property type	<code>void create_property (string property_key, string ↪ property_value)</code>
Label Operations	
Add/remove a label to/from a vertex	<code>void set_node_label (node_id id, string label)</code>
Get all labels of a vertex	<code>std::vector<string> get_label (node_id id)</code>
Add/update/remove property of a vertex	<code>void set_node_property (node_id id, string key, string value)</code>
Get all properties of a vertex	<code>std::vector<string> get_property (node_id id, string key)</code>
Remove all properties of a vertex	<code>void remove_node_property (node_id id, string key)</code>
Add/remove a label to/from an edge	<code>void set_edge_label (node_id src, node_id dst, string label)</code>
Get all labels of an edge	<code>std::vector<string> get_edge_label (node_id src, node_id dst)</code>
Add/update/remove property of an edge	<code>void set_edge_property (node_id src, node_id dst, string key, ↪ string value)</code>
Get all properties of an edge	<code>std::vector<string> get_edge_property (node_id src, node_id ↪ dst, string key)</code>
Remove all properties of an edge	<code>void remove_edge_property (node_id src, node_id dst, string ↪ key)</code>

Table 6.1: Graph API extensions to manage properties and labels in the graph.

workload. The FinBench workload provides queries of varying complexities based on how many properties they access. It is plausible that embedded properties work better for queries that access most attributes, while the split representation might work better for queries that access only a few. We will study how the two approaches perform under these workloads. We will also study how the two approaches perform under different graph sizes and property sizes.

Supporting attributes in the *Edge Key* representation is straightforward. We can store the attributes inline with the vertex/edge (shown in Figure 6.3a) or separately using column stores (shown in Figure 6.3b). Essentially, we need to add new columns to each row to hold the vertex/edge attributes (or just one column to hold all attributes serialized to a raw byte array). Let’s examine how to store properties in the adjacency list representation. We can store the vertex properties inline with the vertex (shown in Figure 6.2a) or separately using column stores (shown in Figure 6.2c). Similarly, we can store the edge properties inline with the out neighbor array (shown in Figure 6.2b) or separately using column stores (shown in Figure 6.2d). If we store properties separately, we need to store additional metadata to map which property belongs to which edge. We call this the offset array, which stores the starting index of each property for each edge. This allows us to have a variable number of properties for each edge.

Regardless of the storage format, each individual edge or vertex’s properties are stored as a raw byte array. We assume a serialization format like Protocol Buffers [14] or FlatBuffers [1] to serialize and deserialize the properties to and from the byte array.

6.4 Label Storage

Our system also needs to support the notion of different vertex labels for realistic graph workloads from LDBC [71]. Edge types implicitly encode the labels of the vertices they connect; for instance, in a social network graph, a "knows" edge can only connect vertices of type "person." However, The LDBC Financial Benchmark schema allows there to be two types of edges connecting nodes of account type – *transfer* and *withdraw*.

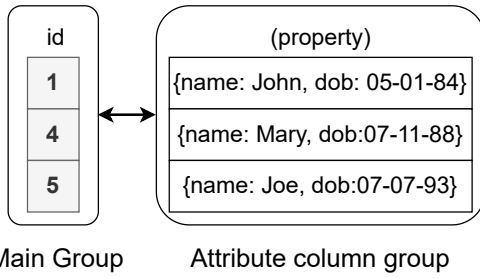
There are a few options to accomplish this: duplicate our Adjacency List and Edge Key representation for each

id	(property)
1	{name: John, dob: 05-01-84}
4	{name: Mary, dob:07-11-88}
5	{name: Joe, dob:07-07-93}

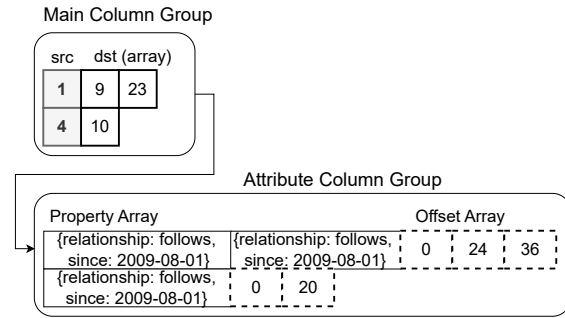
(a) Inline vertex properties in adjacency list

src	dst + properties (array)
1	9 {relationship: follows, since: 2009-08-01} 23 {relationship: follows, since: 2009-08-01}
4	10 {relationship: follows, since: 2009-08-01}

(b) Inline out-edge properties in adjacency list representation



(c) Vertex properties using column groups in adjacency list representation

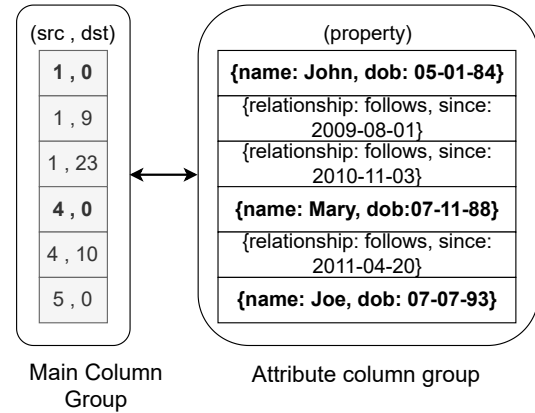


(d) Out-edge properties using column groups in adjacency list representation

Figure 6.2: Two strategies for property storage in the Adjacency List representation. The first strategy stores properties inline with the vertex or edge (Figure 6.2a and Figure 6.2b), while the second strategy stores properties in separate column groups (Figure 6.2c and Figure 6.2d).

(src , dst)	(property)
1 , 0	{name: John, dob: 05-01-84}
1 , 9	{relationship: follows, since: 2009-08-01}
1 , 23	{relationship: follows, since: 2010-11-03}
4 , 0	{name: Mary, dob:07-11-88}
4 , 10	{relationship: follows, since: 2011-04-20}
5 , 0	{relationship: follows, since: 2020-09-12}

(a) Inline properties in EdgeKey



(b) Out-edge properties using column groups in EdgeKey representation

Figure 6.3: Two strategies for property storage in the EdgeKey representation. The first strategy stores properties inline with the vertex or edge (Figure 6.3a), while the second strategy stores properties in separate column groups (Figure 6.3b).

type (label) of vertex. This implies that we are grouping each different type of vertex as a separate graph, with edges possibly going between these “layers” of graphs. Alternatively, we can place vertices of different types within a single representation overall and associate a “label” attribute with each of the vertices. Both approaches could attain the same functionality, but each has its associated problems.

6.4.1 Duplicating Representations

Storing each label in a separate representation is the most straightforward approach. This approach is similar to materializing different views of the graph for each label. However, this approach is not scalable as it complicates our code and is more expensive to query.

Consider the scenario where we have two vertex types: `account` and `business`, and an edge type `transfer` connecting the two. An account can transfer money to a business, and we want to produce the sum of all the `transfer` values to a business from an account with ID x . To do this, we need to query the `transfer` edge table for all edges with source vertex ID x . Then for each edge, we need to query the destination vertex’s label to determine if it is a business. This requires at least one access to another representation’s table which stores the vertex information. This scenario is illustrated in Figure 6.4.

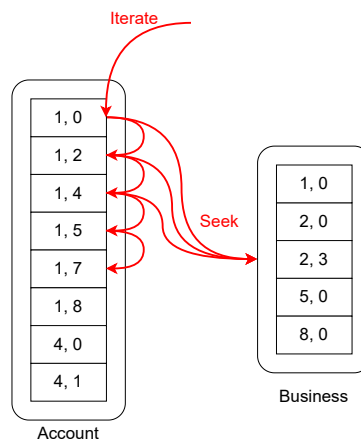


Figure 6.4: Assume that we are maintaining two Edge Key representations, one for Accounts and the other for Businesses. While iterating through the neighborhood of the Account Edge Key table, we have to repeatedly query into the Business Edge Key table. The Adjacency List case is similar.

6.4.2 Embedding Attributes

To avoid the added complexity, another option is to store a “type” attribute in our representations. Associating a “type” with the vertex table still requires an extra lookup of the attribute while we are querying from an edge. Instead, associating a new attribute in the form of “source type - destination type” with each in-/out-neighborhood table in the Adjacency List representation, and the edge table in Edge Key representation can effectively eliminate the extra lookup.

However, having multiple “source type - destination type” pairs requires us to scan the entire neighborhood of a vertex. This situation is shown in Figure 6.5.

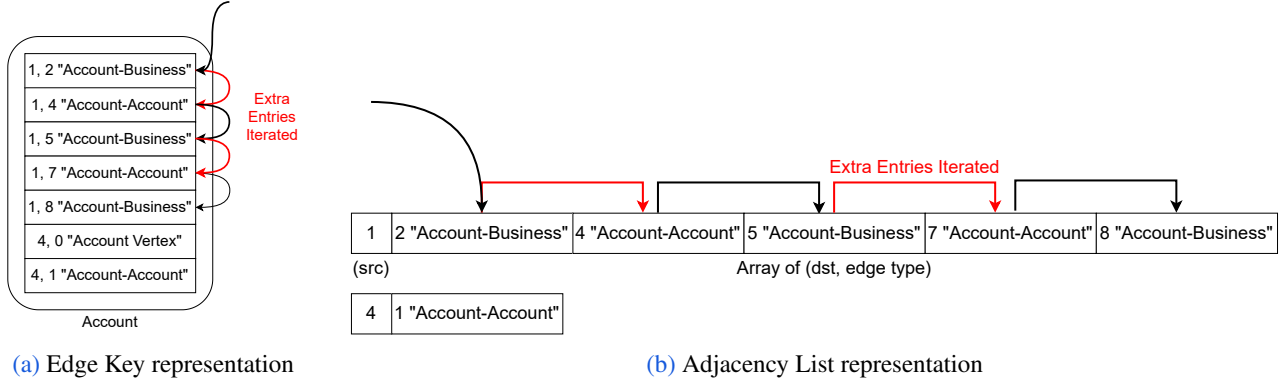


Figure 6.5: Using the “source type - destination type” attribute eliminates the jumping between tables but creates non-sequential accesses in both representations.

6.4.3 Vertex Re-labelling

Having realized the issues underlying these two approaches, we devise a simple method to handle typed node queries: partitioning the vertex ID space according to node type and re-mapping the original vertex IDs. We divide the entire vertex ID space up before all insertions. The vertex ID space is large: from 0 to 2^{32} ¹, so we assume that insertions for vertices of each type will not exceed that partitioned limit.

This not only gives us sequential access when we traverse neighboring vertices of a certain type but also allows us to store no additional columns in the data tables. We only need to maintain a 1-1 mapping between the original vertex IDs and the new vertex IDs that implicitly embeds type information. We present a diagram of this model in figure 6.6.

This change is currently implemented outside Flexograph as a user-space framework on top of the Graph API. We pass the re-labeled vertex IDs into the database and map them back to the original vertex IDs when we obtain vertex ID results. Additionally, we pass partitioned vertex ID ranges to the iterators for them to determine when to pause iteration when we query for specific typed nodes. We assume vertex membership changes are rare – to do so we will need to delete and re-insert the vertex with all its related edges so that it is associated with a newly mapped vertex ID.

6.5 Research Questions

We will evaluate the two property storage strategies under different workloads to answer the following research questions:

1. How does the performance of the two property storage strategies compare under different queries in LDBC Financial Benchmark? Different queries access different and varying numbers of properties, and we will study how the two approaches perform under these workloads.
2. How does the performance of the two property storage strategies compare under different graph sizes and property sizes? We will study how the two approaches perform under different graph sizes and property sizes.

¹can easily be changed to 2^{64} by using 64-bit integers to store IDs

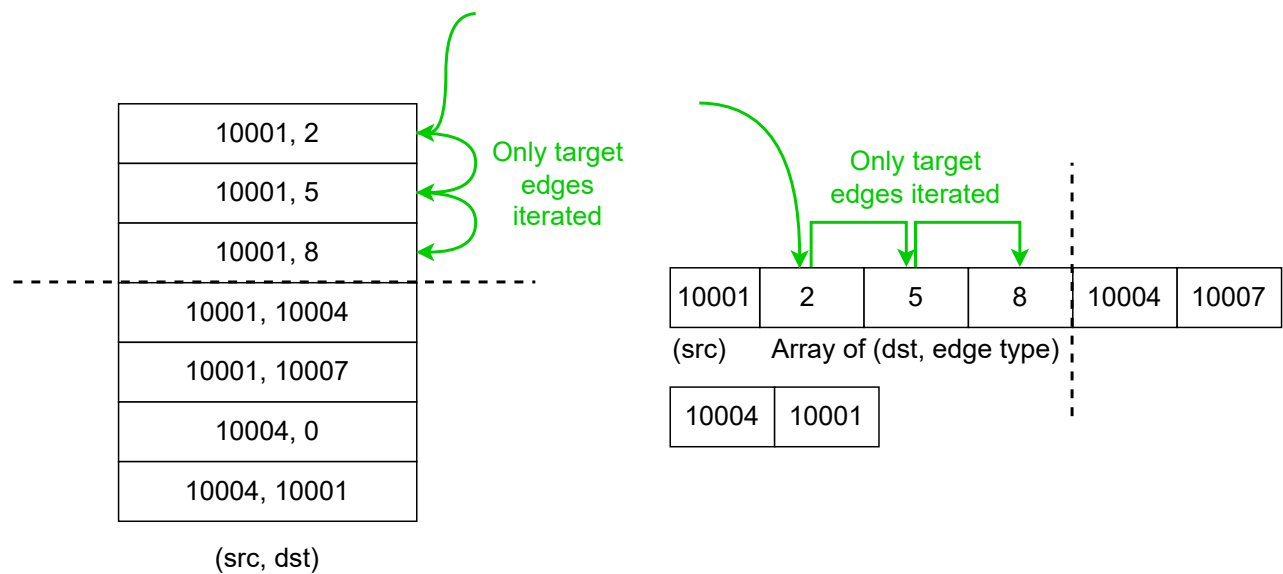


Figure 6.6: Partitioning and re-assigning the vertex id space according to vertex types gives us sequential access without having to scan extra entries in the neighbourhood.

3. What is the slowdown of the two property storage strategies compared to a graph without properties? This lets us determine whether avoiding materializing the structure of the graph is worth the cost of storing properties and the slowdown in query performance.

Chapter 7

Future Work: Specialized Indexes on Graph Data

Most graph databases use indexes to speed up query processing. There can be two types of special indexes that we would want to consider for future extensions of our work. The first type of index allows the user to define a custom index on some higher-order graph structure such as triangles to which each vertex belongs. Such indexes can significantly speed up graph mining queries that are related to clique mining, dense subgraph discovery, clustering, and other graph mining tasks [78].

The second type of index allows the user to define custom traversal patterns that are used to index the graph. For example, a user might wish to access the vertices in a specific order, such as the Hilbert order [64, 89], or based on some property of the vertices.

We believe that these two types of indexes can significantly speed up graph mining queries and are worth exploring in future work. Doing so would require exploring whether this is doable with the WiredTiger indexing framework, and if so, how to implement it.

Chapter 8

Timeline

The timeline for the remainder of the project is shown in Figure 8.1. Several tasks need to be completed before graduation. The first, most important task is to put together a paper on the Flexograph project. We have already started working on this paper, and we expect to submit to VLDB 2025 on September 1st, 2024.

To evaluate Flexograph we need to compare it to other dynamic graph processing systems (Chapter 5) and to do that we need to finish the Grafaree project. The Grafaree project is expected to be completed by the end of August 2024. While the Grafaree project is being completed, we will also be working on introducing the storage of labels and properties in Flexograph. The label and property storage project is expected to be completed by mid-October 2024, and I wish to submit the paper to VLDB 2026 on December 1st, 2025.

I would very much like to do an internship at a company or a research lab, and I am currently looking for opportunities. I would ideally like to work on a project that is related to the Flexograph project, but I am open to other ideas as well. The internship would ideally be in the Spring of 2025, between January and May, giving me ample time to work on writing the dissertation and revise/resubmit the papers.

I wish to graduate in time for the November 2025 convocation and to do that, I need to have my thesis ready by the end of August 2025. The timeline shown in Figure 8.1 is based on the [August 31st program end date doctoral deadline from G+PS](#), but I am flexible with the exact date of graduation. The only hard deadline from G+PS is the dissertation submission deadline, which is July 26th, 2025; I would like to submit the dissertation much before then.

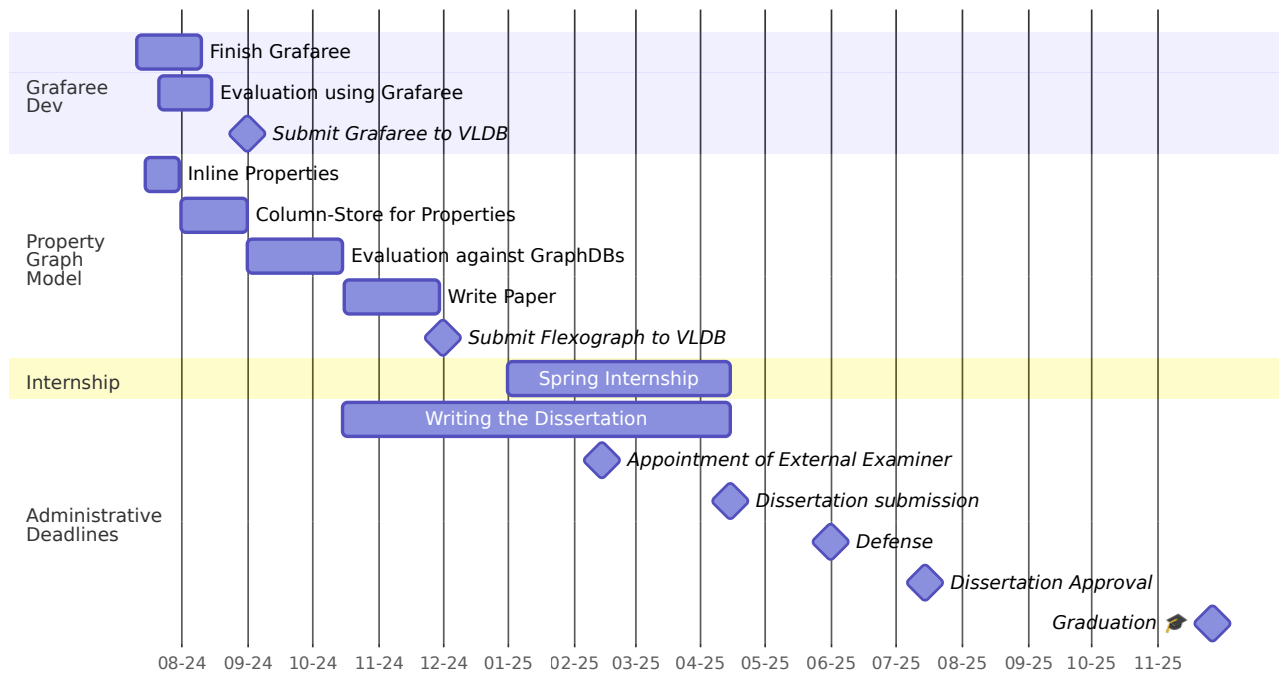


Figure 8.1: Timeline between now until graduation

Bibliography

- [1] Flatbuffers: Flatbuffers. <https://flatbuffers.dev/>. (Accessed on 07/01/2024). → page 41
- [2] Full usa. <http://konect.cc/networks/dimacs9-USA/>. (Accessed on 07/02/2024). → page 36
- [3] sort GNU coreutils. URL https://www.gnu.org/software/coreutils/manual/html_node/sort-invocation.html. → page 32
- [4] Import - operations manual. <https://neo4j.com/docs/operations-manual/current/tools/neo4j-admin/neo4j-admin-import/>. (Accessed on 06/12/2024). → page 32
- [5] Ldbc introduction. <https://ldbcouncil.org/introduction/>. (Accessed on 05/18/2024). → page 9
- [6] Ldbc graphalytics benchmark (ldbc graphalytics). <https://ldbcouncil.org/benchmarks/graphalytics/>, . (Accessed on 05/18/2024). → page 9
- [7] Ldbc social network benchmark (ldbc snb). <https://ldbcouncil.org/benchmarks/snb/>, . (Accessed on 05/18/2024). → page 9
- [8] Wiredtiger: Bulk-load. https://source.wiredtiger.com/11.0.0/tune_bulk_load.html, . (Accessed on 06/12/2024). → page 32
- [9] Wiredtiger overview and features. <http://source.wiredtiger.com/10.0.0/overview.html>, . → pages 5, 25
- [10] Arangodb: Open-source multi-model database for graph, document and search. <https://www.arangodb.com/>. → pages 1, 37
- [11] Apache giraph. <http://giraph.apache.org/>. → page 16
- [12] Graph500 benchmarks. <https://graph500.org/>. → page 3
- [13] Neo4j. <https://neo4j.com/>. → pages 1, 4, 37
- [14] Google Developers | Protocol Buffers. <https://developers.google.com/protocol-buffers>. → page 41
- [15] Vertica. <https://www.vertica.com>. → page 1
- [16] Patent Citation Dataset from SNAP, SHA256: a8ea26e01be2d7d3bff29e4b80e3fac11deb83bb7c691552896e48e9f13098b1. 3.77M Vertices, 16.52M Edges. <https://snap.stanford.edu/data/cit-Patents.html>. → page 36

- [17] C. R. Aberger, A. Lamb, S. Tu, A. Nötzli, K. Olukotun, and C. Ré. Emptyheaded: A relational engine for graph processing. *ACM Trans. Database Syst.*, 42(4), oct 2017. ISSN 0362-5915. doi:10.1145/3129246. URL <https://doi.org/10.1145/3129246>. → page 1
- [18] Z. Ai, M. Zhang, Y. Wu, X. Qian, K. Chen, and W. Zheng. Clip: A disk i/o focused parallel out-of-core graph processing system. *IEEE Transactions on Parallel and Distributed Systems*, 30(1):45–62, 2018. → page 15
- [19] V. Anand, P. Mehrotra, D. Margo, and M. Seltzer. Smooth kronecker: Solving the combing problem in kronecker graphs. In *Proceedings of the 3rd Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–10, 2020. → page 36
- [20] R. Angles. The property graph database model. In *AMW*, 2018. → page 5
- [21] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, and D. Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):1–40, Sept. 2018. ISSN 0360-0300, 1557-7341. doi:10.1145/3104031. → pages 6, 7
- [22] J. Arulraj, A. Pavlo, and P. Menon. Bridging the archipelago between row-stores and column-stores for hybrid workloads. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD '16*, page 583–598, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450335317. doi:10.1145/2882903.2915231. URL <https://doi.org/10.1145/2882903.2915231>. → page 4
- [23] S. Beamer, K. Asanović, and D. Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015. → pages 3, 9, 15, 22, 37
- [24] S. Beamer, K. Asanovic, and D. A. Patterson. The GAP benchmark suite. *CoRR*, abs/1508.03619, 2015. URL <http://arxiv.org/abs/1508.03619>. → page 35
- [25] S. Beamer III. *Understanding and improving graph algorithm performance*. University of California, Berkeley, 2016. → pages 2, 9
- [26] M. Besta, R. Gerstenberger, E. Peter, M. Fischer, M. Podstawski, C. Barthels, G. Alonso, and T. Hoefler. Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries. *ACM Comput. Surv.*, 56(2), sep 2023. ISSN 0360-0300. doi:10.1145/3604932. URL <https://doi.org/10.1145/3604932>. → pages 1, 8, 9
- [27] J. Biskup and H. Stiefeling. Transitive closure algorithms for very large databases. In J. van Leeuwen, editor, *Graph-theoretic concepts in computer science*, page 122–147, Berlin, Heidelberg, 1989. Springer Berlin Heidelberg. ISBN 978-3-540-46076-3. → page 1
- [28] P. A. Boncz, M. Zukowski, and N. Nes. Monetdb/x100: Hyper-pipelining query execution. In *Cidr*, volume 5, pages 225–237. Citeseer, 2005. → page 1
- [29] A. Bonifati, G. Fletcher, H. Voigt, N. Yakovets, and H. Jagadish. *Querying graphs*, volume 10. Springer, 2018. → page 7
- [30] A. Breuer, R. Eilat, and U. Weinsberg. Friend or faux: Graph-based early detection of fake accounts on social networks. In *Proceedings of The Web Conference 2020*, pages 1287–1297, 2020. → page 2

- [31] N. Bronson, Z. Amsden, G. Cabrera, P. Chakka, P. Dimov, H. Ding, J. Ferris, A. Giardullo, S. Kulkarni, H. Li, et al. TAO: Facebook’s distributed data store for the social graph. In *2013 USENIX Annual Technical Conference USENIX ATC 13*), pages 49–60, 2013. → page 1
- [32] M. Capotă, T. Hegeman, A. Iosup, A. Prat-Pérez, O. Erling, and P. Boncz. Graphalytics: A big data benchmark for graph-processing platforms. In *Proceedings of the GRADES’15*, page 7. ACM, 2015. → pages 3, 8
- [33] M. Cha, H. Haddadi, F. Benevenuto, and K. P. Gummadi. Measuring User Influence in Twitter: The Million Follower Fallacy. In *In Proceedings of the 4th International AAAI Conference on Weblogs and Social Media (ICWSM)*, Washington DC, USA, May 2010. → page 36
- [34] D. Chakrabarti and C. Faloutsos. Graph mining: Laws, generators, and algorithms. *ACM Comput. Surv.*, 38(1):2–es, jun 2006. ISSN 0360-0300. doi:10.1145/1132952.1132954. URL <https://doi.org/10.1145/1132952.1132954>. → page 10
- [35] R. Chen, J. Shi, Y. Chen, B. Zang, H. Guan, and H. Chen. Powerlyra: Differentiated graph computation and partitioning on skewed graphs. *ACM Trans. Parallel Comput.*, 5(3), jan 2019. ISSN 2329-4949. doi:10.1145/3298989. URL <https://doi.org/10.1145/3298989>. → pages 1, 15
- [36] M. Curtiss, I. Becker, T. Bosman, S. Doroshenko, L. Grijincu, T. Jackson, S. Kunnatur, S. Lassen, P. Pronin, S. Sankar, et al. Unicorn: A system for searching the social graph. *Proceedings of the VLDB Endowment*, 6(11):1150–1161, 2013. → page 14
- [37] D. De Leo and P. Boncz. Teseo and the analysis of structural dynamic graphs. *Proceedings of the VLDB Endowment*, 14(6):1053–1066, 2021. → pages 14, 18, 35
- [38] L. Dhulipala, G. E. Blelloch, and J. Shun. Low-latency graph streaming using compressed purely-functional trees. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019*, page 918–934, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367127. doi:10.1145/3314221.3314598. URL <https://doi.org/10.1145/3314221.3314598>. → page 18
- [39] R. Diestel and R. Diestel. Graph theory. *Graph Theory*, pages 1–34, 2017. → page 6
- [40] D. Ediger, R. McColl, J. Riedy, and D. A. Bader. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing*, pages 1–5. IEEE, 2012. → pages 18, 35
- [41] N. Elyasi, C. Choi, and A. Sivasubramaniam. {Large-Scale} graph processing on emerging storage devices. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 309–316, 2019. → page 15
- [42] S. Firmli, V. Trigonakis, J.-P. Lozi, I. Psaroudakis, A. Weld, D. Chiadmi, S. Hong, and H. Chafi. Csr++: A fast, scalable, update-friendly graph data structure. In *24th International Conference on Principles of Distributed Systems (OPODIS’20)*, 2020. → page 12
- [43] P. Fuchs, D. Margan, and J. Giceva. Sortledton: a universal, transactional graph data structure. *Proceedings of the VLDB Endowment*, 15(6):1173–1186, 2022. → pages 12, 18

- [44] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In *Presented as part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, pages 17–30, 2012. → pages 15, 17
- [45] J. E. Gonzalez, R. S. Xin, A. Dave, D. Crankshaw, M. J. Franklin, and I. Stoica. Graphx: Graph processing in a distributed dataflow framework. In *OSDI*, volume 14, pages 599–613, 2014. → pages 1, 15, 17
- [46] A. Gupta, D. Agarwal, D. Tan, J. Kulesza, R. Pathak, S. Stefani, and V. Srinivasan. Amazon redshift and the case for simpler data warehouses. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, page 1917–1923, Melbourne Victoria Australia, May 2015. ACM. ISBN 978-1-4503-2758-9. doi:10.1145/2723372.2742795. URL <https://dl.acm.org/doi/10.1145/2723372.2742795>. tex.ids= guptaAmazonRedshiftCase2015. → page 1
- [47] P. Gupta, A. Mhedhbi, and S. Salihoglu. Columnar storage and list-based processing for graph database management systems. *Proc. VLDB Endow.*, 14(11):2491–2504, jul 2021. ISSN 2150-8097. doi:10.14778/3476249.3476297. URL <https://doi.org/10.14778/3476249.3476297>. → pages 4, 39
- [48] R. Hart. Elon musk feuds with yann lecun over the billionaire recruiting for his ai startup. <https://www.forbes.com/sites/roberthart/2024/05/28/elon-musk-is-feuding-with-ai-godfather-yann-lecun-again-heres-why/?sh=5cbfb5542b8b>, May 2024. (Accessed on 06/05/2024). → page 2
- [49] S. Heidari, Y. Simmhan, R. N. Calheiros, and R. Buyya. Scalable graph processing frameworks: A taxonomy and open challenges. *ACM Comput. Surv.*, 51(3):60:1–60:53, June 2018. ISSN 0360-0300. doi:10.1145/3199523. URL <http://doi.acm.org/10.1145/3199523>. → page 15
- [50] R. Kallman, H. Kimura, J. Natkins, A. Pavlo, A. Rasin, S. Zdonik, E. P. C. Jones, S. Madden, M. Stonebraker, Y. Zhang, J. Hugg, and D. J. Abadi. H-store: a high-performance, distributed main memory transaction processing system. *Proc. VLDB Endow.*, 1(2):1496–1499, aug 2008. ISSN 2150-8097. doi:10.14778/1454159.1454211. URL <https://doi.org/10.14778/1454159.1454211>. → page 1
- [51] J. Kim and S. Swanson. Blaze: fast graph processing on fast ssds. In *2022 SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 617–631. IEEE Computer Society, 2022. → pages 15, 37
- [52] P. Kumar and H. H. Huang. Graphone: A data store for real-time analytics on evolving graphs. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 249–263, Boston, MA, Feb. 2019. USENIX Association. ISBN 978-1-939133-09-0. URL <https://www.usenix.org/conference/fast19/presentation/kumar>. → pages 18, 35
- [53] A. Kyrola, G. E. Blelloch, and C. Guestrin. Graphchi: Large-scale graph computation on just a pc. In *OSDI. USENIX*, 2012. → pages 1, 15, 16, 37, 38
- [54] K. Lakhotia, R. Kannan, S. Pati, and V. Prasanna. Gpop: A scalable cache-and memory-efficient framework for graph processing over parts. *ACM Transactions on Parallel Computing (TOPC)*, 7(1):1–24, 2020. → page 15
- [55] M. Lissandrini, M. Brugnara, and Y. Velegrakis. Graph database benchmark. <https://graphbenchmark.com/>. URL <https://doi.org/10.14778/3297753.3297759>. (Accessed on 07/02/2024). → page 37

- [56] Y. Low, D. Bickson, J. Gonzalez, C. Guestrin, A. Kyrola, and J. M. Hellerstein. Distributed graphlab: a framework for machine learning and data mining in the cloud. *Proceedings of the VLDB Endowment*, 5(8): 716–727, 2012. → pages 15, 17
- [57] Y. Low, J. E. Gonzalez, A. Kyrola, D. Bickson, C. E. Guestrin, and J. Hellerstein. Graphlab: A new framework for parallel machine learning. *arXiv preprint arXiv:1408.2041*, 2014. → pages vi, 17
- [58] S. Maass, C. Min, S. Kashyap, W. Kang, M. Kumar, and T. Kim. Mosaic: Processing a trillion-edge graph on a single machine. In *Proceedings of the Twelfth European Conference on Computer Systems*, pages 527–543, 2017. → pages vi, 17
- [59] P. Macko, V. J. Marathe, D. W. Margo, and M. I. Seltzer. LLAMA: Efficient graph analytics using Large Multiversioned Arrays. In *2015 IEEE 31st International Conference on Data Engineering*, 2015. → pages 12, 13, 18
- [60] K. Madduri and D. A. Bader. Compact graph representations and parallel connectivity algorithms for massive dynamic network analysis. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–11. IEEE, 2009. → page 12
- [61] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146. ACM, 2010. → pages 1, 15, 16
- [62] M. Mariappan and K. Vora. Graphbolt: Dependency-driven synchronous processing of streaming graphs. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–16, 2019. → page 18
- [63] R. R. McCune, T. Weninger, and G. Madey. Thinking like a vertex: A survey of vertex-centric frameworks for large-scale distributed graph processing. *ACM Comput. Surv.*, 48(2), oct 2015. ISSN 0360-0300. doi:10.1145/2818185. URL <https://doi.org/10.1145/2818185>. → page 15
- [64] F. McSherry, M. Isard, and D. G. Murray. Scalability! but at what cost? In *HotOS*, volume 15, pages 14–14. Citeseer, 2015. → pages 2, 3, 15, 24, 46
- [65] P. Mehrotra, V. Anand, D. Margo, M. R. Hajidehi, and M. Seltzer. Sok: The faults in our graph benchmarks, 2024. → pages 2, 3, 35
- [66] Neo4j. Understanding neo4j’s data on disk - knowledge base. <https://neo4j.com/developer/kb/understanding-data-on-disk/>. (Accessed on 04/16/2024). → page 4
- [67] M. E. Newman. Power laws, pareto distributions and zipf’s law. *Contemporary physics*, 46(5):323–351, 2005. → page 10
- [68] D. Nguyen, A. Lenharth, and K. Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 456–471. ACM, 2013. → page 15
- [69] A. Pacaci, A. Zhou, J. Lin, and M. T. Özsu. Do we need specialized graph databases? benchmarking real-time social networking applications. GRADES’17, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450350389. doi:10.1145/3078447.3078459. URL <https://doi.org/10.1145/3078447.3078459>. → page 4

- [70] K. Pingali, D. Nguyen, M. Kulkarni, M. Burtscher, M. A. Hassaan, R. Kaleem, T.-H. Lee, A. Lenharth, R. Manevich, M. Méndez-Lojo, et al. The tao of parallelism in algorithms. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*, pages 12–25, 2011. → page 15
- [71] S. Qi, H. Lin, Z. Guo, G. Szárnyas, B. Tong, Y. Zhou, B. Yang, J. Zhang, Z. Wang, Y. Shen, C. Wang, P. Peiravi, H. Gabb, and B. Steer. The ldbc financial benchmark, 2023. URL <https://arxiv.org/abs/2306.15975>. → pages vi, 39, 40, 41
- [72] X. Qiu, W. Cen, Z. Qian, Y. Peng, Y. Zhang, X. Lin, and J. Zhou. Real-time constrained cycle detection in large dynamic graphs. *Proceedings of the VLDB Endowment*, 11(12):1876–1888, 2018. this paper talks about the 20ms latency standard for cycle detection tasks in financial fraud detection systems. → page 2
- [73] M. Raasveldt and H. Mühleisen. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data*, pages 1981–1984, 2019. → page 1
- [74] M. A. Rodriguez and P. Neubauer. The graph traversal pattern. (arXiv:1004.1001), Apr. 2010. URL <http://arxiv.org/abs/1004.1001>. arXiv:1004.1001 [cs]. → page 14
- [75] A. Roy, I. Mihailovic, and W. Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488. ACM, 2013. → pages 15, 37
- [76] Y. Saad. *Iterative methods for sparse linear systems*. SIAM, 2003. → page 12
- [77] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proc. VLDB Endow.*, 11(4):420–431, Dec. 2017. ISSN 2150-8097. doi:10.1145/3164135.3164139. URL <https://doi.org/10.1145/3164135.3164139>. → page 3
- [78] Y. Sasaki, G. Fletcher, and O. Makoto. Language-aware indexing for conjunctive path queries. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pages 661–673. IEEE, 2022. → page 46
- [79] D. Sengupta, N. Sundaram, X. Zhu, T. L. Willke, J. Young, M. Wolf, and K. Schwan. Graphin: An online high performance incremental graph processing framework. In *Euro-Par 2016: Parallel Processing: 22nd International Conference on Parallel and Distributed Computing, Grenoble, France, August 24-26, 2016, Proceedings 22*, pages 319–333. Springer, 2016. → page 18
- [80] S. Shen, Z. Yao, L. Shi, L. Wang, L. Lai, Q. Tao, L. Su, R. Chen, W. Yu, H. Chen, B. Zang, and J. Zhou. Bridging the gap between relational OLTP and graph-based OLAP. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 181–196, Boston, MA, July 2023. USENIX Association. ISBN 978-1-939133-35-9. URL <https://www.usenix.org/conference/atc23/presentation/shen>. → page 4
- [81] X. Shi, B. Cui, Y. Shao, and Y. Tong. Tornado: A system for real-time iterative analysis over evolving data. In *Proceedings of the 2016 International Conference on Management of Data*, pages 417–430, 2016. → page 18
- [82] J. Shun and G. E. Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *ACM Sigplan Notices*, volume 48, pages 135–146. ACM, 2013. → pages 1, 15, 37
- [83] SNAP. Snap: Network datasets: Epinions social network. <https://snap.stanford.edu/data/soc-Epinions1.html>. (Accessed on 05/01/2024). → page 10

- [84] C. Staudt, A. Sazonovs, and H. Meyerhenke. Networkkit: An interactive tool suite for high-performance network analysis. *arXiv preprint arXiv:1403.3005*, 2014. → page 12
- [85] M. Stonebraker and A. Weisberg. The voltdb main memory dbms. *IEEE Data Eng. Bull.*, 36(2):21–27, 2013. → page 1
- [86] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. O’Neil, et al. C-store: a column-oriented dbms. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, pages 491–518. 2018. → page 1
- [87] N. Sundaram, N. Satish, M. M. A. Patwary, S. R. Dulloor, M. J. Anderson, S. G. Vadlamudi, D. Das, and P. Dubey. Graphmat: High performance graph analytics made productive. *Proc. VLDB Endow.*, 8(11): 1214–1225, July 2015. ISSN 2150-8097. doi:10.14778/2809974.2809983. URL <https://doi.org/10.14778/2809974.2809983>. → page 15
- [88] C. Tang, Y. Li, Z. Luo, M. Ghosh, H. Wu, L. Zhang, A. Lu, R. Kabra, N. K. Navadiya, P. Mishra, et al. Taming hybrid-cloud fast and scalable graph analytics at twitter. In *GLOBECOM 2022-2022 IEEE Global Communications Conference*, pages 498–503. IEEE, 2022. → page 2
- [89] A. Trostanovsky. *Vertex-and-edge ordering for faster parallel graph processing*. PhD thesis, University of British Columbia, 2023. → pages 24, 46
- [90] L. G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990. → page 16
- [91] K. Vora. {LUMOS}:{Dependency-Driven} disk-based graph processing. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 429–442, 2019. → page 37
- [92] K. Vora, R. Gupta, and G. Xu. Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations. In *Proceedings of the twenty-second international conference on architectural support for programming languages and operating systems*, pages 237–251, 2017. → page 18
- [93] B. Wheatman and H. Xu. Packed compressed sparse row: A dynamic graph representation. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*, pages 1–7, 2018. doi:10.1109/HPEC.2018.8547566. → page 12
- [94] C. Xie, R. Chen, H. Guan, B. Zang, and H. Chen. Sync or async: time to fuse for distributed graph-parallel computation. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP 2015, page 194–204, New York, NY, USA, Jan. 2015. Association for Computing Machinery. ISBN 978-1-4503-3205-7. doi:10.1145/2688500.2688508. URL <https://doi.org/10.1145/2688500.2688508>. → pages vi, 17
- [95] E. P. Xing, Q. Ho, W. Dai, J.-K. Kim, J. Wei, S. Lee, X. Zheng, P. Xie, A. Kumar, and Y. Yu. Petuum: A new platform for distributed machine learning on big data. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1335–1344, 2015. → page 15
- [96] W. Yang, T. Li, G. Fang, and H. Wei. Pase: Postgresql ultra-high-dimensional approximate nearest neighbor search extension. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’20, page 2241–2253, New York, NY, USA, 2020. Association for Computing Machinery.

ISBN 9781450367356. doi:10.1145/3318464.3386131. URL <https://doi.org/10.1145/3318464.3386131>. → page 1

- [97] M. Zhang, Y. Wu, Y. Zhuo, X. Qian, C. Huan, and K. Chen. Wonderland: A novel abstraction-based out-of-core graph processing system. *ACM SIGPLAN Notices*, 53(2):608–621, 2018. → page 15
- [98] D. Zheng, D. Mhembere, R. Burns, J. Vogelstein, C. E. Priebe, and A. S. Szalay. {FlashGraph}: Processing {Billion-Node} graphs on an array of commodity {SSDs}. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 45–58, 2015. → page 15
- [99] X. Zhu, W. Han, and W. Chen. {GridGraph}:{Large-Scale} graph processing on a single machine using 2-level hierarchical partitioning. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 375–386, 2015. → page 15
- [100] X. Zhu, W. Chen, W. Zheng, and X. Ma. Gemini: A computation-centric distributed graph processing system. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, pages 301–316, 2016. → page 15
- [101] X. Zhu, G. Feng, M. Serafini, X. Ma, J. Yu, L. Xie, A. Aboulnaga, and W. Chen. Livegraph: A transactional graph storage system with purely sequential adjacency list scans. *arXiv preprint arXiv:1910.05773*, 2019. → pages 18, 35

Chapter 9

Appendix

9.1 PageRank Kernel Implementation using GraphAPI

```
1  pvector<ScoreT> pagerank(GraphEngine& graph_engine,
2                          int thread_num,
3                          int max_iters,
4                          node_id_t num_nodes,
5                          node_id_t max_node_id,
6                          double epsilon = 0)
7  {
8      pvector<ScoreT> src(max_node_id, 0);
9      pvector<ScoreT> dst(max_node_id, 1 / num_nodes);
10     pvector<node_id_t> deg(max_node_id, 0);
11
12     #pragma omp parallel for
13     for (int i = 0; i < thread_num; i++)
14     {
15         GraphBase* graph = graph_engine.create_graph_handle();
16         NodeCursor* node_cursor = graph->get_node_iter();
17         node_cursor->set_key_range(graph_engine.get_key_range(i));
18
19         node found = {0};
20         node_cursor->next(&found);
21
22         while (found.id != -1)
23         {
24             deg[found.id] = found.out_degree;
25             node_cursor->next(&found);
26         }
27         node_cursor->close();
28         graph->close(false);
29     }
30
31     for (int iter = 0; iter < max_iters; iter++)
32     {
```

```

33     double error = 0;
34 #pragma omp parallel for reduction(+ : error)
35     for (int i = 0; i < thread_num; i++)
36     {
37         GraphBase* graph = graph_engine.create_graph_handle();
38         InCursor* in_cursor = graph->get_innbd_iter();
39         in_cursor->set_key_range(graph_engine.get_key_range(i));
40
41         adjlist found;
42         in_cursor->next(&found);
43
44         while (found.node_id != -1)
45         {
46             ScoreT incoming_total = 0;
47             for (node_id_t v : found.edgelist)
48             {
49                 incoming_total += src[v];
50             }
51
52             ScoreT old_score = dst[found.node_id];
53             dst[found.node_id] =
54                 (1 - kDamp) / num_nodes + kDamp * incoming_total;
55             error += fabs(dst[found.node_id] - old_score);
56             src[found.node_id] = dst[found.node_id] / deg[found.node_id];
57
58             in_cursor->next(&found);
59         }
60
61         in_cursor->close();
62         graph->close(false);
63     }
64     printf(" %2d    %lf\n", iter, error);
65     if (error < epsilon) break;
66 }
67 return dst;
68 }

```

Listing 9.1: GAPBS PageRank kernel modified to use GraphAPI

Let us examine, for instance, our modifications to the GAPBS PageRank implementation to use GraphAPI, as shown in [Listing 9.1](#). The basic structure of the PageRank implementation is unchanged from the original GAPBS, but with changes introduced to use the iterator classes and the GraphAPI. To use OpenMP construct `parallel for`, usually one can just add a pragma directive before the loop, and the compiler will automatically parallelize the loop. However, automatic parallelization based on the loop construct is not possible in our system because we need each thread to construct its private graph handle and iterator. Additionally, automatic parallelization does not allow us to statically assign iterator ranges to the threads as we do in the `calculate_thread_offsets()` method.

To remedy this, we have to manually parallelize the loop and assign each thread a range of nodes to work on. We use the `calculate_thread_offsets()` method to calculate the key range for each thread during GraphEngine instantiation. In the OpenMP parallel block, each thread uses the `get_key_range()` method to get the key range for itself.

GAPBS uses a few utility classes that it uses to store algorithm state and updates. The `pvector` is a modified vector class that allows parallel initializations and resizing. C++ standard library's `std::vector` is not thread-safe and always serially initializes the elements when resizing. `pvector`, on the other hand, initializes a new vector in parallel but does not initialize the elements when resizing. `sliding_queue` is a double-buffered queue class that allows atomic appends to a buffer that are not visible until the `SlideWindow()` method is called.

These utility classes are memory-based and are destroyed when the application is closed. Flexograph, however, does not wish to lose any algorithmic state that is stored in these utility classes, and strives to provide a mechanism to persist this state to disk for future use. To do so, we modified the `pvector` and `sliding_queue` classes to use an `mmap` region as their internal buffer for storing the state. The basic functionality remains unchanged, however, the destructor of these classes now calls the `msync()` system call to persist the state to disk.